

AN INTRODUCTION TO SYSTEMVERILOG

SystemVerilog for VHDL Designers

The Example Cookbook

One CORDIC Rotator · Two Languages · Six Testbenches



Luigi Giuffrida, Alessandra Dolmeta

Department of Electronics and Telecommunications
Politecnico di Torino

Draft — September 17, 2026

This booklet is one of four companions: Design, Verification, Simulation, and this worked Example. A CORDIC rotator, small enough to read in one sitting, is developed in VHDL and in SystemVerilog and verified with six testbenches in three languages. Every source file is reproduced in full and distributed alongside this document under [code/cordic/](#); every one of them has been analysed and simulated before being typeset.

Preface

Reading rules is not the same as reading code. This booklet is one design, worked all the way through: a CORDIC rotator, in VHDL and in SystemVerilog, verified by six testbenches in three languages. Every file is reproduced in full, not excerpted.

This is the **Example Cookbook**, one of four companions covering SystemVerilog for VHDL Designers. It assumes the material of the *Design Cookbook*, the *Verification Cookbook* and the *Simulation Cookbook*, and applies all three, in one place, to one design.

What this booklet is

Chapter 1 develops `cordic_rot` from its specification and fixed-point format, through a VHDL implementation and a SystemVerilog one, to six testbenches: a plain VHDL testbench, a class-based SystemVerilog testbench, a UVM environment, a C++ testbench driven through Verilator, and two Python testbenches, one talking to the design over a socket and one written with cocotb. Every construct used has already been introduced somewhere in the companion booklets; here you see it used together with everything else, under time pressure, in a design with real corners: a state machine, a parameterised datapath, a fixed-point number format, and a handshake protocol.

The code

All the sources are distributed with this document under `code/cordic/`. Every one of them has been analysed and simulated before being typeset, and every line of tool output quoted in this booklet was produced by actually running something. Where a construct does not work in a particular tool, the text says which tool and which version.

Acknowledgement

The choice of a CORDIC rotator as the worked example is not accidental. It is small enough to read in one sitting, large enough to need a state machine, a parameterised datapath, a fixed-point format and a handshake protocol, and arithmetic enough that a golden model in Python earns its keep. If you understand every line of this booklet, you can write SystemVerilog.

How to Read This Booklet

Typographic conventions

- Monospaced text is code: an identifier, a keyword, a file name or a command.
- Green monospaced text is a file in the accompanying source tree.
- Sans-serif names are tools: QuestaSim, Verilator, cocotb, GTKWave.

Listings that carry line numbers are complete, reproduced in full from `code/cordic/`; nothing here is excerpted.

How to read this one

Read it with the sources open in an editor, not on their own. Chapter 1 is written as one continuous build: specification, fixed-point format, the VHDL implementation, the SystemVerilog implementation, then testbench after testbench. It assumes you already have the vocabulary from the *Design Cookbook*, the *Verification Cookbook* and the *Simulation Cookbook*; where a construct is used here without re-explanation, that is where it was introduced.

If you only want to see one testbench style, the chapter is written so that each testbench section is self-contained once the design itself is understood: you do not have to read all six to follow one.

The companion booklets

This is the **Example Cookbook**. the *Design Cookbook* and the *Verification Cookbook* cover the language used here; the *Simulation Cookbook* covers the tools (Verilator, QuestaSim, cocotb) that run the testbenches built here.

Contents

Preface	v
How to Read This Booklet	vii
1 A Complete Example: A CORDIC Rotator	1
1.1 What We Are Building, and Why This Example	1
1.2 The Algorithm	2
1.3 Choosing the Number Format	4
1.4 The Golden Model Comes First	6
1.5 The VHDL Implementation	9
1.6 The SystemVerilog Implementation	20
1.7 The Two Implementations Compared	30
1.8 The VHDL Testbench	31
1.9 The SystemVerilog Testbench	39
1.10 The C++ Testbenches	55
1.11 The Python Testbench	75
1.12 Build and Run	84
1.13 What Is Not Verified	87
1.14 Exercises	88
1.15 Checklist	88
Further Reading	91
Index	93

A Complete Example: A CORDIC Rotator

One design of real, if modest, complexity, written twice. This chapter contains the complete VHDL and SystemVerilog sources for an iterative CORDIC rotator, four testbenches for it in three languages, a Python golden model, and the evidence that all of them agree. Every file is reproduced in full, and every line of output quoted here was produced by actually running the tools.

1.1 What We Are Building, and Why This Example

The block is `cordic_rot`: a rotation-mode CORDIC that takes a vector (x, y) and an angle θ and returns the vector rotated by θ , scaled by a fixed gain. Feed it $x = \text{X0_UNIT}$, $y = 0$ and it hands you $(\cos \theta, \sin \theta)$.

That is a small specification. What makes it the right example for this book is everything the implementation needs in order to satisfy it:

- a **fixed-point number format** that somebody has to choose, and that neither language will check for you in the same way;
- a **constant table** computed at elaboration from a transcendental function, which is where VHDL's `ieee.math_real` and SystemVerilog's `$atan` meet;
- a **control state machine**, small enough to read on one page;
- a **parameterised datapath** with shifts, adds and a saturating output stage;
- a **handshake protocol** on both ports, which is what makes a proper monitor and scoreboard necessary rather than decorative;
- and **arithmetic** whose correct answer is not obvious by inspection, so a golden model earns its keep.

If you understand every line of this chapter, you can write SystemVerilog. There is nothing else in the language you will meet in your first year that is not a variation on something here.

Figure 1.1 is the whole block. Three stages and a controller: a coarse rotation that folds the angle into the range where the algorithm converges, a micro-rotation datapath that iterates N times, and a saturating narrow-down on the way out.

Folded, not pipelined

This is the area-efficient shape: one set of adders, reused N times, at a throughput of one transaction every $N + 2$ cycles. The other shape unrolls the loop into N pipeline stages and accepts one transaction per cycle at roughly N times the area. Section 1.14 asks you to build that version, because the interesting part is not the datapath but what happens to the handshake.

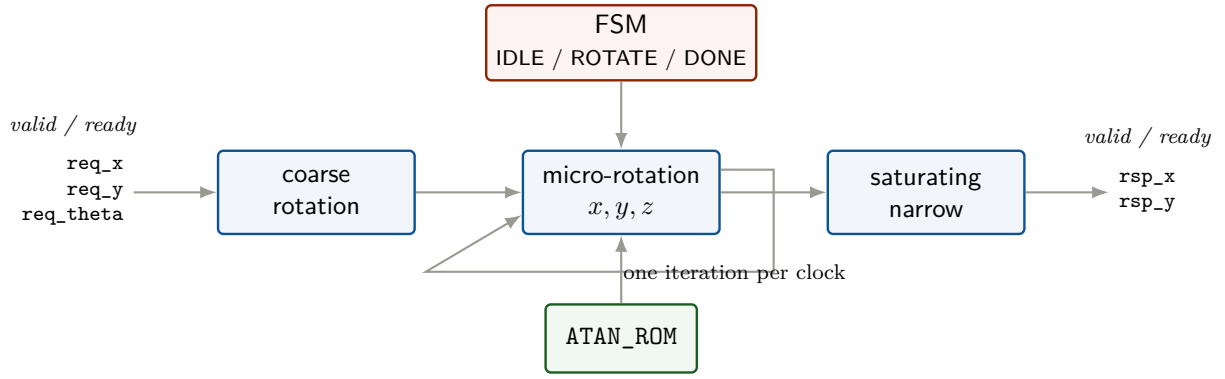


Figure 1.1. The `cordic_rot` datapath. The coarse-rotation stage runs once per transaction; the micro-rotation stage runs N times, reusing one adder pair and one shifter pair. A transaction takes $N + 2$ clock cycles.

1.2 The Algorithm

A rotation by θ is

$$\begin{pmatrix} x' \\ y' \end{pmatrix} = \begin{pmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix},$$

which needs four multiplications and two transcendental functions. Volder's observation in 1959 was that if you restrict yourself to rotations by the particular angles $\alpha_i = \arctan 2^{-i}$, the matrix becomes

$$\begin{pmatrix} x_{i+1} \\ y_{i+1} \end{pmatrix} = \cos \alpha_i \begin{pmatrix} 1 & -d_i 2^{-i} \\ d_i 2^{-i} & 1 \end{pmatrix} \begin{pmatrix} x_i \\ y_i \end{pmatrix}, \quad d_i = \pm 1,$$

and the only operations left inside the matrix are a shift and an add. Drop the $\cos \alpha_i$ scale factor, keep a running account of how much angle is left to rotate, and you have the algorithm:

$$x_{i+1} = x_i - d_i (y_i \gg i) \tag{1.1}$$

$$y_{i+1} = y_i + d_i (x_i \gg i) \tag{1.2}$$

$$z_{i+1} = z_i - d_i \alpha_i \tag{1.3}$$

with $z_0 = \theta$ and $d_i = +1$ if $z_i \geq 0$, -1 otherwise. The decision is the sign bit of z_i : no comparison, no subtraction, one wire.

Three consequences follow, and all three appear in the code.

1.2.1 The processing gain

Dropping $\cos \alpha_i$ at every step means the vector grows. After N iterations the length has been multiplied by

$$K_N = \prod_{i=0}^{N-1} \sqrt{1 + 2^{-2i}},$$

which converges very quickly: $K_{14} = 1.6467602540$ and the limit is $K_\infty = 1.6467602581$. The gain is a constant, so you can cancel it once, in advance, by pre-scaling the input. The package exposes `X0_UNIT = round(214/K) = 9949` for exactly this purpose.

Which K ?

The testbenches use $K = 1.6467602581210656$, the limit, while the true gain for $N = 14$ is 1.6467602540312922 . The difference is 4×10^{-9} , about 6.7×10^{-5} of one LSB in Q1.14, so it is invisible next to the 2.5 LSB of real quantisation error. Using the limit is the conventional choice because it does not change when you change N . If you ever tighten the tolerance below one LSB, use the exact product for your N .

1.2.2 The convergence range

The angle left over after all N steps can only be driven to zero if the step you are about to take is never larger than the sum of all the steps remaining. That gives

$$|\theta| \leq \sum_{i=0}^{\infty} \arctan 2^{-i} = 1.7433 \text{ rad} \approx 99.9^\circ.$$

So the raw recurrence handles a little under $\pm 100^\circ$, and nothing more.

1.2.3 Quadrant folding

To cover the full circle you rotate by a whole quadrant first, which costs nothing because a rotation by $\pm\pi/2$ is a swap and a negation:

$$+\frac{\pi}{2} : (x, y) \mapsto (-y, x), \quad -\frac{\pi}{2} : (x, y) \mapsto (y, -x).$$

One fold brings $|\theta| \leq \pi$ down to $|\theta| \leq \pi/2 = 1.5708$, comfortably inside 1.7433. Two folds would cover $|\theta| \leq 3\pi/2$, and the design does not do them: it accepts $|\theta| \leq \pi$ and fires an assertion otherwise. Range reduction of an arbitrary angle belongs upstream, where it can be done once for a whole stream.

Figure 1.2 shows what the micro-rotations actually do. Each one overshoots, and the overshoot halves.

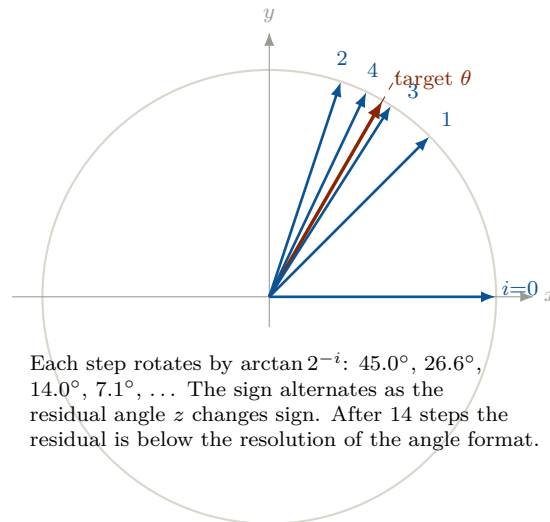


Figure 1.2. The first four micro-rotations converging on a target of 60° . The vector also grows by the processing gain at every step; that growth is not drawn.

The table of angles is fixed at elaboration. Table 1.1 is the one this design uses, taken verbatim from the generated file `sv/cordic_atan_rom.svh`.

Table 1.1. The arctangent ROM. Values are in the internal angle format, Q2.15, so one unit is $2^{-15} = 3.05 \times 10^{-5}$ rad.

i	$\arctan 2^{-i}$ (rad)	Q2.15	i	$\arctan 2^{-i}$ (rad)	Q2.15
0	0.785398163	25736	7	0.007812341	256
1	0.463647609	15193	8	0.003906230	128
2	0.244978663	8027	9	0.001953123	64
3	0.124354995	4075	10	0.000976562	32
4	0.062418810	2045	11	0.000488281	16
5	0.031239833	1024	12	0.000244141	8
6	0.015623729	512	13	0.000122070	4

Where the table stops paying

From $i = 5$ onwards the entries are exactly 2^{15-i} : the arctangent has become indistinguishable from its argument at this precision. That is the signal that $N = 14$ is matched to a 15-bit fractional angle, and that adding a fifteenth iteration would buy nothing without widening the angle first.

1.3 Choosing the Number Format

This is the section where a VHDL designer discovers what SystemVerilog will not do for them.

1.3.1 The decisions

x and y : signed 16 bits, 14 fractional (Q1.14). The natural output is a unit vector, so the range needs to cover $[-1, 1]$ with a little room. One integer bit gives $[-2, 2]$ and a resolution of $2^{-14} = 6.1 \times 10^{-5}$. Sixteen bits because that is what fits a DSP block input and what the rest of the world will hand you.

θ : signed 16 bits, 13 fractional (Q2.13). The angle has to reach $\pi = 3.14159$, so two integer bits are the minimum; the range is $[-4, 4]$ and the resolution is 1.22×10^{-4} rad. Note that this is *coarser* than the output resolution: an angle error of 1.22×10^{-4} rad produces an output error of up to 1.22×10^{-4} , which is two LSB of Q1.14. The input quantisation, not the algorithm, is the dominant error term.

Guard bits: two at each end. GL = 2 extra fractional bits below the LSB stop the fourteen successive truncations in Equations (1.1) and (1.2) from accumulating into the result; each shift throws away up to one LSB, and fourteen of them would otherwise cost about four. GM = 2 extra bits above the MSB give the datapath room for the processing gain: an input of magnitude 2 becomes 3.3 inside the loop, and without headroom it would wrap.

The output saturates rather than wraps. A CORDIC that silently returns -1.99 when the answer was $+2.01$ is worse than one that returns $+1.99$, because the wrap looks like a sign error and takes an afternoon to find.

Table 1.2 collects every constant.

Table 1.2. Every number format in the design. The internal formats are the external ones widened by GL at the bottom and GM at the top.

Quantity	Type	Bits	Frac.	Range	Resolution
x, y in/out	<code>data_t</code>	16	14	$[-2, 2)$	6.1×10^{-5}
θ in	<code>angle_t</code>	16	13	$[-4, 4)$	1.22×10^{-4}
x, y internal	<code>idata_t</code>	20	16	$[-8, 8)$	1.5×10^{-5}
z internal	<code>iangle_t</code>	18	15	$[-4, 4)$	3.05×10^{-5}
<i>Derived constants</i>					
<code>PI_EXT</code>	$\pi \cdot 2^{13}$		25736	the accepted input limit	
<code>HALF_PI_INT</code>	$\frac{\pi}{2} \cdot 2^{15}$		51472	the coarse-rotation threshold	
<code>XO_UNIT</code>	$2^{14}/K$		9949	the unit-circle seed	
<code>MAX_DATA</code>	$2^{15} - 1$		32767	the saturation limit	

1.3.2 The error budget

Three terms, and it is worth being able to write them down before you simulate, so that a measured error you did not predict is a signal rather than a shrug.

1. **Angle quantisation.** $\pm \frac{1}{2}$ LSB of Q2.13, so 6.1×10^{-5} rad, giving up to 1.0 LSB of Q1.14 output error.
2. **Residual angle after N steps.** Bounded by $\alpha_{N-1} = 1.22 \times 10^{-4}$ rad, another 2.0 LSB, but in practice much less because the residual is roughly uniform in $[-\alpha_{N-1}, \alpha_{N-1}]$.
3. **Truncation in the shifts.** Fourteen right shifts, each losing up to one internal LSB of 2^{-16} , and the guard bits mean those internal LSBs are a quarter of an output LSB. Call it 1 LSB.

Sum: under 5 LSB. Measured, over 308 random and directed transactions: 2.55 LSB in the VHDL testbench, 2.82 in the SystemVerilog one. The tolerance in every testbench is set to 12 LSB, which leaves room for the tails without being loose enough to hide a wrong sign, a wrong shift or a missing micro-rotation. All three of those bugs move the answer by hundreds of LSB.

△ Trap: Nobody is tracking your binary point

VHDL-2008 ships `ieee.fixed_pkg`. It gives you `sfixed(1 downto -14)`, and when you multiply an `sfixed(1 downto -14)` by an `sfixed(3 downto -13)` the compiler works out that the result is an `sfixed(5 downto -27)` and tells you so. Shifts, resizes and the binary-point bookkeeping are all checked at analysis time.

SystemVerilog has nothing of the kind. `data_t` is twenty bits and the binary point exists only in Table 1.2, in the comments, and in your head. Every one of the decisions above is a comment that the tool will never verify. This is the single largest thing a VHDL designer gives up when moving to SystemVerilog for arithmetic work, and the mitigation is entirely procedural: name the formats in the package, write the widths in the comments, and check them against a golden model that was written independently.

Write the number format down before you write the RTL, put it in the package header, and have a model that will disagree with you if you get it wrong.

1.4 The Golden Model Comes First

The Python model in Listing 1.1 was written before either implementation, and both were checked against it.

Two things about it are deliberate.

It is **bit-exact**, not floating point. A floating point model tells you the design computes a rotation; a bit-exact model tells you the design computes *this* rotation, with this rounding, these guard bits and this saturation. The second catches a class of bug the first cannot see: a shift that rounds the wrong way, a guard bit dropped one stage too early, a saturation limit off by one. Both checks appear in this chapter, and they fail on different things.

It exploits the fact that **Python's » on a negative integer floors**, which is exactly what an arithmetic shift right does in hardware. $-7 \gg 1$ is -4 in Python and in VHDL's `shift_right` and in SystemVerilog's `»>`. It is -3 in C, which rounds toward zero. Writing the model in Python rather than C removed a whole category of mismatch before it could happen.

△ Trap: A model that copies the RTL is worth nothing

The temptation, once the RTL exists, is to write the model by reading it. The result agrees with the RTL perfectly, including on every bug. The model here was written from Equations (1.1) to (1.3) and the format table, and the first time it was run against the VHDL it disagreed. The VHDL was wrong. That is the entire value proposition.

Running the model on its own checks it against the ideal rotation:

```
$ make model
N = 14, K = 1.646760, 1/K = 0.607253, X0_UNIT = 9949
HALF_PI = 51472, PI = 102944
atan table: [25736, 15193, 8027, 4075, 2045, 1024, 512, 256, 128, 64, 32,
             16, 8, 4]
worst |error| over [-pi, pi] = 0.000233 (3.82 LSB)
```

3.82 LSB over a 721-point sweep of the full circle, against a budget of 5. The model is behaving as the arithmetic says it should, which means it is fit to be the reference.

```

1  #!/usr/bin/env python3
2  """
3  Bit-exact golden model of the ``cordic_rot`` core, plus the floating point
4  reference it is checked against.
5
6  The model is deliberately written so that every line maps onto one line of
7  RTL. It is the single source of truth used by
8
9  * the SystemVerilog testbench (through vectors written by gen_vectors.py),
10 * the Verilator C++ testbench (through a generated header),
11 * the cocotb testsuite (imported directly).
12
13 Number formats
14 -----
15 x, y      signed, DW bits, QF fractional bits   (Q1.14 -> [-2, 2) )
16 theta     signed, AW bits, QA fractional bits   (Q2.13 -> [-4, 4) )
17
18 internal x, y signed, IW = DW + GL + GM bits, QF + GL fractional bits
19 internal z  signed, IA = AW + GL bits, QA + GL fractional bits
20
21 ``GL`` adds precision at the bottom, ``GM`` adds headroom at the top so that
22 the CORDIC processing gain cannot overflow the datapath.
23 """
24
25 import math
26
27 # -----
28 # Parameters -- keep in step with cordic_pkg.sv / cordic_pkg.vhd
29 # -----
30 DW = 16      # external x/y width
31 QF = 14      # external x/y fractional bits
32 AW = 16      # external angle width
33 QA = 13      # external angle fractional bits
34 N = 14       # number of CORDIC micro-rotations
35 GL = 2       # guard bits added at the LSB side
36 GM = 2       # guard bits added at the MSB side
37
38 IW = DW + GL + GM      # internal x/y width          (20)
39 IQF = QF + GL          # internal x/y fractional bits (16)
40 IA = AW + GL           # internal angle width         (18)
41 IQA = QA + GL          # internal angle frac. bits    (15)
42
43 HALF_PI = round(math.pi / 2 * (1 << IQA))      # 51472
44 PI_INT = round(math.pi * (1 << IQA))             # 102944
45
46
47 def atan_table():
48     """arctan(2**-i) in the internal angle format, i = 0 .. N-1."""
49     return [round(math.atan(2.0 ** -i) * (1 << IQA)) for i in range(N)]
50
51
52 ATAN = atan_table()
53
54
55 def cordic_gain():
56     """The processing gain K = prod sqrt(1 + 2**-2i)."""
57     k = 1.0
58     for i in range(N):
59         k *= math.sqrt(1.0 + 2.0 ** (-2 * i))
60     return k
61
62
63 K = cordic_gain()                                     # ~1.64676

```

```

64 INV_K = 1.0 / K                                # ~0.60725
65 XO_UNIT = round(INV_K * (1 << QF))              # 9949, the "unit circle" seed
66
67
68 # -----
69 # Small fixed point helpers
70 # -----
71 def to_signed(value, width):
72     """Interpret the low `width` bits of `value` as two's complement."""
73     value &= (1 << width) - 1
74     if value & (1 << (width - 1)):
75         value -= 1 << width
76     return value
77
78
79 def wrap(value, width):
80     """Truncate to `width` bits, keeping the two's complement interpretation."""
81     return to_signed(value, width)
82
83
84 def asr(value, shift):
85     """Arithmetic shift right. Python's >> on a negative int already floors,
86     which is exactly what a hardware arithmetic shift does."""
87     return value >> shift
88
89
90 def saturate(value, width):
91     """Clamp `value` to the range of a signed `width`-bit number."""
92     lo = -(1 << (width - 1))
93     hi = (1 << (width - 1)) - 1
94     return max(lo, min(hi, value))
95
96
97 def q_to_float(value, frac):
98     return value / float(1 << frac)
99
100
101 def float_to_q(value, frac, width):
102     return saturate(int(round(value * (1 << frac))), width)
103
104
105 # -----
106 # The bit-exact model
107 # -----
108 def cordic_rot(x0, y0, theta, trace=False):
109     """Rotate the vector (x0, y0) by `theta`.
110
111     Arguments and results are *integers* in the external formats described at
112     the top of this file. Returns (x, y).
113     """
114     # ---- 1. widen into the internal format -----
115     x = x0 << GL
116     y = y0 << GL
117     z = theta << GL
118
119     # ---- 2. coarse rotation: bring z into the convergence range -----
120     # The micro-rotation series converges for |z| <= 1.7433 rad, so an angle
121     # outside +-pi/2 is first rotated by a whole quadrant. Rotating by +pi/2
122     # maps (x, y) -> (-y, x); rotating by -pi/2 maps (x, y) -> (y, -x).
123     if z > HALF_PI:
124         x, y, z = -y, x, z - HALF_PI
125     elif z < -HALF_PI:
126         x, y, z = y, -x, z + HALF_PI

```

```

127
128 # ---- 3. the micro-rotation loop -----
129 for i in range(N):
130     dx = asr(y, i)
131     dy = asr(x, i)
132     if z >= 0:                                # rotate counter-clockwise
133         xn, yn, zn = x - dx, y + dy, z - ATAN[i]
134     else:                                      # rotate clockwise
135         xn, yn, zn = x + dx, y - dy, z + ATAN[i]
136     x, y, z = wrap(xn, IW), wrap(yn, IW), wrap(zn, IA)
137     if trace:
138         print(f" i={i:2d} x={x:8d} y={y:8d} z={z:8d}")
139
140 # ---- 4. narrow back down, with saturation -----
141 return saturate(asr(x, GL), DW), saturate(asr(y, GL), DW)
142
143
144 def cordic_float(x0, y0, theta):
145     """Floating point reference, including the CORDIC processing gain."""
146     c, s = math.cos(theta), math.sin(theta)
147     return K * (x0 * c - y0 * s), K * (x0 * s + y0 * c)
148
149
150 def sincos(theta_rad):
151     """Convenience wrapper: cos/sin of an angle given in radians."""
152     theta = float_to_q(theta_rad, QA, AW)
153     x, y = cordic_rot(X0_UNIT, 0, theta)
154     return q_to_float(x, QF), q_to_float(y, QF)
155
156
157 # -----
158 # Self-check
159 # -----
160 if __name__ == "__main__":
161     print(f"N = {N}, K = {K:.6f}, 1/K = {INV_K:.6f}, X0_UNIT = {X0_UNIT}")
162     print(f"HALF_PI = {HALF_PI}, PI = {PI_INT}")
163     print("atan table:", ATAN)
164
165     worst = 0.0
166     steps = 721
167     for k in range(steps):
168         ang = -math.pi + 2 * math.pi * k / (steps - 1)
169         ang = max(-math.pi, min(math.pi, ang))
170         c, s = sincos(ang)
171         err = max(abs(c - math.cos(ang)), abs(s - math.sin(ang)))
172         worst = max(worst, err)
173     print(f"worst |error| over [-pi, pi] = {worst:.6f}"
174           f" ({worst * (1 << QF):.2f} LSB)")

```

Listing 1.1. `cordic_model.py` — the bit-exact golden model. Every line maps onto one line of RTL.

1.5 The VHDL Implementation

Two files: a package that holds the formats, the types and the table, and an entity that holds the logic. The complete sources are Listings 1.2 and 1.3 at the end of this section; the pieces worth stopping on are here.

1.5.1 The package

The subtypes are the first thing a reader meets, and they carry real information:

```
1 subtype data_t is signed(DW - 1 downto 0);
2 subtype angle_t is signed(AW - 1 downto 0);
3 subtype idata_t is signed(IW - 1 downto 0);
4 subtype iangle_t is signed(IA - 1 downto 0);
```

A subtype constrains an existing type. `data_t` and `signed(15 downto 0)` are interchangeable, and every assignment between them is checked for width and for direction. Assign an `idata_t` to a `data_t` and the analyser stops you.

The arctangent table is computed by the analyser:

```
1 function build_atan_rom return atan_rom_t is
2   variable rom : atan_rom_t;
3 begin
4   for i in 0 to N - 1 loop
5     rom(i) := to_signed(
6       integer(round(arctan(2.0 ** (-i)) * 2.0 ** IQA)), IA);
7   end loop;
8   return rom;
9 end function build_atan_rom;
```

`arctan` and `round` come from `ieee.math_real`, which is a simulation and elaboration library: the call is evaluated once, when the constant is elaborated, and only the fourteen integers reach the netlist. Table 1.1 is literally the output of this function.

The constant itself is *deferred*: the package declares that it exists and the package body says what it is.

```
1 -- in the package declaration
2 constant ATAN_ROM : atan_rom_t;
3 -- in the package body, after build_atan_rom
4 constant ATAN_ROM : atan_rom_t := build_atan_rom;
```

Recipe: Use a deferred constant to initialise from a function

Writing `constant ATAN_ROM : atan_rom_t := build_atan_rom;` in the package declaration works, but it calls the function from the declaration while the body that defines it has not been elaborated, and analysers warn about that. The deferred form removes the ambiguity and costs one line. SystemVerilog has no equivalent: a `localparam` must be given its value where it is declared.

The saturating narrow-down is the last piece of arithmetic in the design:

```
1 function sat_narrow (v : idata_t) return data_t is
2   constant TW : positive := IW - GL;
3   variable t : signed(TW - 1 downto 0);
4 begin
5   t := v(IW - 1 downto GL);           -- drop the guard bits
6   if t > to_signed(MAX_DATA, TW) then
7     return to_signed(MAX_DATA, DW);
8   elsif t < to_signed(MIN_DATA, TW) then
9     return to_signed(MIN_DATA, DW);
10  else
11    return resize(t, DW);
12  end if;
13 end function sat_narrow;
```

Slicing off the bottom GL bits of a `signed` is exactly an arithmetic shift right, and it rounds toward $-\infty$, matching the Python model.

1.5.2 The architecture

The coarse rotation is one process (`all`):

```

1  x_ext <= shift_left(resize(req_x_i, IW), GL);
2  z_ext <= shift_left(resize(req_theta_i, IA), GL);
3
4  p_coarse : process (all) is
5  begin
6      if z_ext > HALF_PI_S then
7          x_load <= -y_ext; y_load <= x_ext;
8          z_load <= z_ext - HALF_PI_S;
9      elsif z_ext < -HALF_PI_S then
10         x_load <= y_ext; y_load <= -x_ext;
11         z_load <= z_ext + HALF_PI_S;
12     else
13         x_load <= x_ext; y_load <= y_ext; z_load <= z_ext;
14     end if;
15 end process p_coarse;

```

`resize` on a `signed` sign-extends; `shift_left` adds the guard bits. Both are explicit, both are checked. If `IW` and the width of `x_ext` ever disagree, the analyser says so.

The micro-rotation is four concurrent assignments, which is as close as VHDL gets to writing down Equations (1.1) to (1.3) directly:

```

1  ccw    <= not z_q(IA - 1);      -- z >= 0
2  x_sh   <= shift_right(x_q, iter_q);
3  y_sh   <= shift_right(y_q, iter_q);
4  atan_i <= ATAN_ROM(iter_q);
5
6  x_rot <= x_q - y_sh when ccw = '1' else x_q + y_sh;
7  y_rot <= y_q + x_sh when ccw = '1' else y_q - x_sh;
8  z_rot <= z_q - atan_i when ccw = '1' else z_q + atan_i;

```

`shift_right` on a `signed` is arithmetic by definition. There is no way to write the wrong one.

iter_q is a constrained integer

`signal iter_q : natural range 0 to ITER - 1;` declares a counter that cannot legally hold 14. If a bug ever drove it out of range, the simulation would stop with a bounds error naming the line. The SystemVerilog counter is `logic [3:0]` and would wrap to zero in silence.

```

1  -- =====
2  -- cordic_pkg.vhd -- shared declarations for the CORDIC rotator.
3  --
4  -- The VHDL counterpart of cordic_pkg.sv. Read the two side by side: the
5  -- content is identical, the packaging is not.
6  --
7  -- Number formats
8  -- -----
9  --     x, y      signed, DW bits, QF fractional bits   (Q1.14 -> [-2, 2) )
10 --     theta     signed, AW bits, QA fractional bits   (Q2.13 -> [-4, 4) )
11 --
12 -- Internally the datapath is widened by GL bits at the bottom (precision)
13 -- and GM bits at the top (headroom for the CORDIC processing gain).
14 --
15 -- Compile with: vcom -2008 cordic_pkg.vhd
16 -- =====
17 library ieee;
18 use ieee.std_logic_1164.all;
19 use ieee.numeric_std.all;
20 use ieee.math_real.all;           -- arctan, round: elaboration-time only
21
22 package cordic_pkg is
23
24     -- -----
25     -- Format constants
26     -- -----
27     constant DW : positive := 16;    -- external x/y width
28     constant QF : natural  := 14;    -- external x/y fractional bits
29     constant AW : positive := 16;    -- external angle width
30     constant QA : natural  := 13;    -- external angle fractional bits
31     constant N  : positive := 14;    -- number of micro-rotations
32     constant GL : natural  := 2;     -- guard bits at the LSB side
33     constant GM : natural  := 2;     -- guard bits at the MSB side
34
35     constant IW : positive := DW + GL + GM; -- internal x/y width      (20)
36     constant IQF : natural := QF + GL;     -- internal x/y frac. bits (16)
37     constant IA : positive := AW + GL;     -- internal angle width   (18)
38     constant IQA : natural := QA + GL;     -- internal angle frac.   (15)
39
40     -- -----
41     -- Subtypes. A VHDL subtype constrains an existing type; it does not
42     -- create a new one, so `data_t' and `signed(15 downto 0)' are freely
43     -- interchangeable and the compiler still checks the direction and the
44     -- bounds of every assignment.
45     -- -----
46     subtype data_t    is signed(DW - 1 downto 0);
47     subtype angle_t   is signed(AW - 1 downto 0);
48     subtype idata_t   is signed(IW - 1 downto 0);
49     subtype iangle_t  is signed(IA - 1 downto 0);
50
51     -- A record is a genuine composite type: it cannot be treated as a bit
52     -- vector, and it cannot cross a port boundary of a different type.
53     type cordic_req_t is record
54         x      : data_t;
55         y      : data_t;
56         theta  : angle_t;
57     end record cordic_req_t;
58
59     type cordic_rsp_t is record
60         x : data_t;
61         y : data_t;
62     end record cordic_rsp_t;
63

```

```

64  -- -----
65  -- Elaboration-time constants
66  -- -----
67  constant MAX_DATA : integer := 2 ** (DW - 1) - 1;    -- 32767
68  constant MIN_DATA : integer := -2 ** (DW - 1);       -- -32768
69
70  constant HALF_PI_INT : integer := 51472;            -- round(pi/2 * 2**IQA)
71  constant PI_INT      : integer := 102944;           -- round(pi   * 2**IQA)
72  constant PI_EXT      : integer := 25736;            -- round(pi   * 2**QA)
73
74  -- 2**QF / K, with K = prod sqrt(1 + 2**(-2i)) the CORDIC processing gain.
75  constant XO_UNIT : integer := 9949;
76
77  -- -----
78  -- The arctangent ROM, computed by the analyser. ieee.math_real is a
79  -- simulation/elaboration library: the function call below is evaluated
80  -- once, when the constant is elaborated, and only the resulting integers
81  -- reach the netlist.
82  -- -----
83  type atan_rom_t is array (0 to N - 1) of iangle_t;
84
85  function build_atan_rom return atan_rom_t;
86
87  -- A *deferred* constant: the package declares that it exists, the package
88  -- body says what its value is. This is the idiomatic way to initialise a
89  -- constant from a function declared in the same package, and it keeps the
90  -- elaboration order unambiguous. SystemVerilog has no equivalent -- a
91  -- 'localparam' must be given its value where it is declared.
92  constant ATAN_ROM : atan_rom_t;
93
94  -- -----
95  -- Narrow the internal datapath back to the external format, with
96  -- saturation.
97  -- -----
98  function sat_narrow (v : idata_t) return data_t;
99
100  -- Testbench helpers: fixed point <-> real.
101  function q_to_real (v : integer; f : natural) return real;
102  function real_to_q (v : real;    f : natural) return integer;
103
104  end package cordic_pkg;
105
106
107  package body cordic_pkg is
108
109  function build_atan_rom return atan_rom_t is
110      variable rom : atan_rom_t;
111  begin
112      for i in 0 to N - 1 loop
113          rom(i) := to_signed(
114              integer(round(arctan(2.0 ** (-i)) * 2.0 ** IQA)), IA);
115      end loop;
116      return rom;
117  end function build_atan_rom;
118
119  constant ATAN_ROM : atan_rom_t := build_atan_rom;
120
121  function sat_narrow (v : idata_t) return data_t is
122      constant TW : positive := IW - GL;
123      variable t  : signed(TW - 1 downto 0);
124  begin
125      -- Dropping the GL guard bits is an arithmetic shift right, i.e. a plain
126      -- slice of the upper bits. It rounds towards minus infinity, exactly

```

```
127  -- like the Python golden model.
128  t := v(IW - 1 downto GL);
129  if t > to_signed(MAX_DATA, TW) then
130    return to_signed(MAX_DATA, DW);
131  elsif t < to_signed(MIN_DATA, TW) then
132    return to_signed(MIN_DATA, DW);
133  else
134    return resize(t, DW);
135  end if;
136  end function sat_narrow;
137
138  function q_to_real (v : integer; f : natural) return real is
139  begin
140    return real(v) / 2.0 ** f;
141  end function q_to_real;
142
143  function real_to_q (v : real; f : natural) return integer is
144  begin
145    return integer(round(v * 2.0 ** f));
146  end function real_to_q;
147
148  end package body cordic_pkg;
```

Listing 1.2. `cordic_pkg.vhd` — formats, types, the elaboration-time arctangent ROM and the saturating narrow-down.

```

1  -- =====
2  -- cordic_rot.vhd -- iterative (folded) CORDIC rotator, rotation mode.
3  --
4  -- Rotates the vector (x, y) by theta and returns
5  --
6  --       $x' = K * (x * \cos(\theta) - y * \sin(\theta))$ 
7  --       $y' = K * (x * \sin(\theta) + y * \cos(\theta))$ 
8  --
9  -- with  $K \sim 1.6468$  the CORDIC processing gain. Seed the core with
10 --  $x = \text{work.cordic\_pkg.XO\_UNIT}$ ,  $y = 0$  to obtain  $(\cos \theta, \sin \theta)$ .
11 --
12 -- One micro-rotation per clock cycle; a transaction takes  $N + 2$  cycles.
13 -- Request and response use an independent valid/ready handshake.
14 --
15 -- Accepted input range:  $|\theta| \leq \pi$ .
16 --
17 -- Compile with: vcom -2008 cordic_pkg.vhd cordic_rot.vhd
18 -- =====
19 library ieee;
20     use ieee.std_logic_1164.all;
21     use ieee.numeric_std.all;
22
23 library work;
24     use work.cordic_pkg.all;
25
26 entity cordic_rot is
27     generic (
28         -- The VHDL generic and the SystemVerilog parameter play the same role.
29         -- The difference is that a generic is *typed*: `positive' here rules out
30         -- zero and negative values at analysis time.
31         ITER : positive := N
32     );
33     port (
34         clk_i      : in  std_logic;
35         rst_ni     : in  std_logic;      -- asynchronous, active low
36
37         -- ---- request channel ----
38         req_valid_i : in  std_logic;
39         req_ready_o : out std_logic;
40         req_x_i     : in  data_t;
41         req_y_i     : in  data_t;
42         req_theta_i : in  angle_t;
43
44         -- ---- response channel ----
45         rsp_valid_o : out std_logic;
46         rsp_ready_i : in  std_logic;
47         rsp_x_o     : out data_t;
48         rsp_y_o     : out data_t
49     );
50 end entity cordic_rot;
51
52
53 architecture rtl of cordic_rot is
54
55     -- -----
56     -- The state type. A VHDL enumeration has no encoding of its own: the
57     -- synthesis tool picks one (or is told to by an attribute). In
58     -- SystemVerilog you write the base type yourself.
59     -- -----
60     type state_t is (ST_IDLE, ST_ROTATE, ST_DONE);
61
62     signal state_q, state_d : state_t;
63

```

```

64  signal x_q, x_d : idata_t;
65  signal y_q, y_d : idata_t;
66  signal z_q, z_d : iangle_t;
67  signal iter_q, iter_d : natural range 0 to ITER - 1;
68
69  -- Stage 1 -- widen and coarse-rotate
70  signal x_ext, y_ext, x_load, y_load : idata_t;
71  signal z_ext, z_load                : iangle_t;
72
73  -- Stage 2 -- one micro-rotation
74  signal x_sh, y_sh, x_rot, y_rot : idata_t;
75  signal z_rot, atan_i            : iangle_t;
76  signal ccw                      : std_logic;
77
78  constant HALF_PI_S : iangle_t := to_signed(HALF_PI_INT, IA);
79
80  begin
81
82  -----
83  -- Stage 1: widen and coarse-rotate.
84  --
85  -- The micro-rotation series converges only for |z| <= 1.7433 rad, so an
86  -- angle outside +-pi/2 is first rotated by a whole quadrant:
87  --   +pi/2 : (x, y) -> (-y,  x)
88  --   -pi/2 : (x, y) -> ( y, -x)
89  -- Both are exact.
90  --
91  -- 'resize' on a 'signed' sign-extends; 'shift_left' adds the guard bits.
92  -- Note how much more explicit this is than the SystemVerilog cast: the
93  -- compiler will refuse the assignment if the widths do not match.
94  -----
95  x_ext <= shift_left(resize(req_x_i,    IW), GL);
96  y_ext <= shift_left(resize(req_y_i,    IW), GL);
97  z_ext <= shift_left(resize(req_theta_i, IA), GL);
98
99  p_coarse : process (all) is
100  begin
101      if z_ext > HALF_PI_S then
102          x_load <= -y_ext;
103          y_load <=  x_ext;
104          z_load <=  z_ext - HALF_PI_S;
105      elsif z_ext < -HALF_PI_S then
106          x_load <=  y_ext;
107          y_load <= -x_ext;
108          z_load <=  z_ext + HALF_PI_S;
109      else
110          x_load <=  x_ext;
111          y_load <=  y_ext;
112          z_load <=  z_ext;
113      end if;
114  end process p_coarse;
115
116  -----
117  -- Stage 2: one micro-rotation, purely combinational.
118  --
119  --      d      = sign(z)
120  --      x[i+1] = x[i] - d * (y[i] >> i)
121  --      y[i+1] = y[i] + d * (x[i] >> i)
122  --      z[i+1] = z[i] - d * atan(2**-i)
123  --
124  -- 'shift_right' on a 'signed' is arithmetic by definition -- there is no
125  -- way to get it wrong, which is not true of Verilog's '>>' versus '>>>'.
126  -----

```

```

127 ccw    <= not z_q(IA - 1);          -- z >= 0
128 x_sh    <= shift_right(x_q, iter_q);
129 y_sh    <= shift_right(y_q, iter_q);
130 atan_i  <= ATAN_ROM(iter_q);
131
132 x_rot <= x_q - y_sh when ccw = '1' else x_q + y_sh;
133 y_rot <= y_q + x_sh when ccw = '1' else y_q - x_sh;
134 z_rot <= z_q - atan_i when ccw = '1' else z_q + atan_i;
135
136 -----
137 -- Stage 3: the control FSM.
138 --
139 -- `process (all)` (VHDL-2008) is the closest thing VHDL has to
140 -- `always_comb`: the sensitivity list is derived automatically. What it
141 -- does not give you is the latch check -- the default assignments below
142 -- are still your own responsibility.
143 -----
144 p_fsm : process (all) is
145 begin
146     state_d    <= state_q;
147     x_d        <= x_q;
148     y_d        <= y_q;
149     z_d        <= z_q;
150     iter_d     <= iter_q;
151     req_ready_o <= '0';
152     rsp_valid_o <= '0';
153
154     case state_q is
155
156         when ST_IDLE =>
157             req_ready_o <= '1';
158             if req_valid_i = '1' then
159                 x_d    <= x_load;
160                 y_d    <= y_load;
161                 z_d    <= z_load;
162                 iter_d <= 0;
163                 state_d <= ST_ROTATE;
164             end if;
165
166         when ST_ROTATE =>
167             x_d <= x_rot;
168             y_d <= y_rot;
169             z_d <= z_rot;
170             if iter_q = ITER - 1 then
171                 state_d <= ST_DONE;
172             else
173                 iter_d <= iter_q + 1;
174             end if;
175
176         when ST_DONE =>
177             rsp_valid_o <= '1';
178             if rsp_ready_i = '1' then
179                 state_d <= ST_IDLE;
180             end if;
181
182     end case;
183 end process p_fsm;
184
185 -----
186 -- Stage 4: the state registers.
187 -----
188 p_regs : process (clk_i, rst_ni) is
189 begin

```

```

190     if rst_ni = '0' then
191         state_q <= ST_IDLE;
192         x_q     <= (others => '0');
193         y_q     <= (others => '0');
194         z_q     <= (others => '0');
195         iter_q  <= 0;
196     elsif rising_edge(clk_i) then
197         state_q <= state_d;
198         x_q     <= x_d;
199         y_q     <= y_d;
200         z_q     <= z_d;
201         iter_q  <= iter_d;
202     end if;
203 end process p_regs;
204
205 -----
206 -- Outputs
207 -----
208 rsp_x_o <= sat_narrow(x_q);
209 rsp_y_o <= sat_narrow(y_q);
210
211 -----
212 -- Built-in checks. A VHDL `assert' inside a clocked process is the
213 -- equivalent of a SystemVerilog immediate assertion. For the temporal
214 -- property below, plain VHDL has no equivalent of `assert property':
215 -- either you write the state machine by hand, as here, or you use PSL,
216 -- which most simulators accept inside a VHDL comment prefixed `-- psl'.
217 -----
218 -- synthesis translate_off
219 p_check : process (clk_i) is
220     variable prev_valid : std_logic := '0';
221     variable prev_x     : data_t    := (others => '0');
222     variable prev_y     : data_t    := (others => '0');
223     variable prev_theta : angle_t   := (others => '0');
224     variable prev_stall : std_logic := '0';
225 begin
226     if rising_edge(clk_i) and rst_ni = '1' then
227         -- input range
228         if req_valid_i = '1' and req_ready_o = '1' then
229             assert req_theta_i <= to_signed(PI_EXT, AW)
230                 and req_theta_i >= to_signed(-PI_EXT, AW)
231                 report "cordic_rot: |theta| > pi (theta = "
232                     & integer'image(to_integer(req_theta_i)) & ")"
233                 severity error;
234         end if;
235
236         -- a stalled request must keep its payload stable
237         if prev_stall = '1' then
238             assert req_x_i = prev_x and req_y_i = prev_y
239                 and req_theta_i = prev_theta
240                 report "cordic_rot: request payload changed while stalled"
241                 severity error;
242         end if;
243
244         prev_stall := req_valid_i and not req_ready_o;
245         prev_valid := req_valid_i;
246         prev_x     := req_x_i;
247         prev_y     := req_y_i;
248         prev_theta := req_theta_i;
249     end if;
250 end process p_check;
251 -- synthesis translate_on
252

```

```
253 end architecture rtl;
```

Listing 1.3. `cordic_rot.vhd` — the VHDL implementation.

1.6 The SystemVerilog Implementation

The same two files, in the same order, doing the same job. Read the side-by-side pairs below and then the complete sources in Listings 1.4 and 1.5.

1.6.1 Package and types

VHDL

```
package cordic_pkg is
  constant DW : positive := 16;
  subtype data_t is
    signed(DW - 1 downto 0);
  ...
end package cordic_pkg;

-- in the design unit:
library work;
use work.cordic_pkg.all;
```

SystemVerilog

```
package cordic_pkg;
  parameter int unsigned DW = 16;
  typedef logic signed [DW-1:0]
    data_t;
  ...
endpackage : cordic_pkg

// in the design unit:
module cordic_rot
  import cordic_pkg::*;
```

A typedef creates a genuinely new name, but it carries no constraint: `data_t` is sixteen bits interpreted as two's complement, and that is all the tool knows. Assigning a twenty-bit value to it truncates, silently. The import on the module header is the tidy form, and it scopes the import to that module rather than to the compilation unit.

1.6.2 The constant table

VHDL

```
-- declaration
constant ATAN_ROM : atan_rom_t;

-- body
constant ATAN_ROM : atan_rom_t
  := build_atan_rom;
```

SystemVerilog

```
function automatic atan_rom_t
  build_atan_rom();
  ...
endfunction

localparam atan_rom_t ATAN_ROM
  = build_atan_rom();
```

Inside the function, `$atan` plays the part of `arctan` and `$rtoi` the part of `integer(round(...))`:

```
1 | rom[i] = iangle_t'($rtoi($atan(2.0 ** (-i)) * (2.0 ** IQA) + 0.5));
```

`$rtoi` truncates toward zero, so the `+ 0.5` is doing the rounding that VHDL's `round` does for you. Forget it and every entry in the table is one too small, the rotations are all slightly short, and the error grows with the angle: a bug that a tolerance of 12 LSB would not catch at small angles and would catch at large ones.

△ Trap: A constant function is not portable

A function that returns an unpacked array, used to initialise a `localparam`, is legal IEEE 1800 and works in Verilator 5.020 and in the commercial simulators. Icarus Verilog 12 rejects it outright: Array `rom` needs an array index here followed by Unable to evaluate parameter `ATAN_ROM`. Some synthesis front ends refuse `$atan` in a constant expression too. When portability matters, generate the table from a script instead — Section 1.11 does exactly that, and `sv/cordic_atan_rom.svh` is the result.

1.6.3 Widening and coarse rotation

VHDL

```
x_ext <= shift_left(
    resize(req_x_i, IW), GL);
z_ext <= shift_left(
    resize(req_theta_i, IA), GL);
```

SystemVerilog

```
x_ext = idata_t'(req_x_i) <<< GL;
z_ext = iangle_t'(req_theta_i) <<< GL;
```

The cast sign-extends because both types are signed. This is one of the few places where SystemVerilog's implicit width rules genuinely help: the cast states the intent and the shift does the rest, in one line instead of two nested calls. It is also one line in which a typo – `iangle_t` where you meant `idata_t` – compiles happily and truncates two bits.

1.6.4 The micro-rotation

VHDL

```
x_sh <= shift_right(x_q, iter_q);

x_rot <= x_q - y_sh
    when ccw = '1'
    else x_q + y_sh;
```

SystemVerilog

```
x_sh = x_q >>> iter_q;

if (ccw) x_rot = x_q - y_sh;
else    x_rot = x_q + y_sh;
```

△ Trap: » and >> look almost the same

`x_q » iter_q` compiles, simulates, and gives the right answer for every positive input. It is wrong for every negative one, and half of a random stimulus stream is negative, so the failure is loud — if you have a scoreboard. If your testbench prints waveforms for a human to inspect, and the human happens to look at a positive case, this bug ships. VHDL makes it unwriteable.

1.6.5 The state machine

VHDL

```
type state_t is
    (ST_IDLE, ST_ROTATE, ST_DONE);

p_fsm : process (all) is
begin
    state_d <= state_q;
    req_ready_o <= '0';
    case state_q is
        when ST_IDLE =>
            req_ready_o <= '1';
        ...
    end case;
end process p_fsm;
```

SystemVerilog

```
typedef enum logic [1:0] {
    ST_IDLE, ST_ROTATE, ST_DONE
} state_e;

always_comb begin
    state_d = state_q;
    req_ready_o = 1'b0;
    unique case (state_q)
        ST_IDLE: begin
            req_ready_o = 1'b1;
            ...
        endcase
    end
```

Three differences, all of them consequential.

The VHDL enumeration has no encoding: the synthesis tool picks one, and you influence it with an attribute. The SystemVerilog enumeration has an explicit base type, `logic [1:0]`, and that

is what goes into the flip-flops. The SystemVerilog version is more honest about the hardware and less convenient when you want the tool to choose one-hot for you.

The VHDL `case` must cover every value of `state_t` or the analyser complains, and there are exactly three. The SystemVerilog `case` covers three of four possible two-bit values, so it needs a default, and `unique` asks the simulator to check at run time that exactly one branch matched. VHDL gets that check for free from its type system.

`always_comb` is not quite `process (all)`. Both derive the sensitivity list. `always_comb` additionally executes once at time zero, and it includes the contents of any function it calls in the sensitivity list. Neither of them checks for an inferred latch: the default assignments at the top of the block are your responsibility in both languages.

1.6.6 Registers and checks

VHDL

```
p_regs : process (clk_i, rst_ni) is
begin
    if rst_ni = '0' then
        state_q <= ST_IDLE;
        x_q      <= (others => '0');
    elsif rising_edge(clk_i) then
        state_q <= state_d;
        x_q      <= x_d;
    end if;
end process p_regs;
```

SystemVerilog

```
always_ff @(posedge clk_i or
            negedge rst_ni) begin
    if (!rst_ni) begin
        state_q <= ST_IDLE;
        x_q      <= '0;
    end else begin
        state_q <= state_d;
        x_q      <= x_d;
    end
end
```

This is the pair that will convince you the two languages are the same language wearing different clothes. `'0` is the one novelty: an unsized fill literal that takes the width of its context, so it works for a two-bit enumeration and a twenty-bit datapath without being told the width.

The checks are where SystemVerilog pulls ahead. Both designs carry an immediate assertion on the input range:

VHDL

```
assert req_theta_i
    <= to_signed(PI_EXT, AW)
    and req_theta_i
    >= to_signed(-PI_EXT, AW)
report "|theta| > pi"
severity error;
```

SystemVerilog

```
assert (req_theta_i <= angle_t'(PI_EXT) &&
        req_theta_i >= -angle_t'(PI_EXT))
    else $error("cordic_rot: |theta| > pi (%0d)",
               req_theta_i);
```

But the second check — that a stalled request holds its payload steady — is a *temporal* property, and there the two diverge completely. In SystemVerilog it is four lines:

```
1 property p_req_stable;
2   @(posedge clk_i) disable iff (!rst_ni)
3     (req_valid_i && !req_ready_o) | =>
4       $stable(req_x_i) && $stable(req_y_i) && $stable(req_theta_i);
5 endproperty
6 a_req_stable : assert property (p_req_stable)
7   else $error("cordic_rot: request payload changed while stalled");
```

In VHDL it is a process with five variables that remembers the previous cycle by hand, twenty lines in Listing 1.3, or it is PSL, which is a second language with patchy tool support. This is the clearest single win for SystemVerilog in the whole design.

```

1 // =====
2 // cordic_pkg.sv -- shared declarations for the CORDIC rotator.
3 //
4 // This package is the SystemVerilog counterpart of cordic_pkg.vhd. It is
5 // worth reading the two side by side: they contain exactly the same
6 // information, but SystemVerilog lets the design *and* the testbench import
7 // it with a single `import cordic_pkg::*;`, while VHDL needs the design unit
8 // to name the library and the package separately.
9 //
10 // Number formats
11 // -----
12 //   x, y      signed, DW bits, QF fractional bits   (Q1.14 -> [-2, 2) )
13 //   theta     signed, AW bits, QA fractional bits   (Q2.13 -> [-4, 4) )
14 //
15 // Internally the datapath is widened by GL bits at the bottom (precision)
16 // and GM bits at the top (headroom for the CORDIC processing gain).
17 // =====
18 // verilator lint_off UNUSEDPARAM
19 // (several constants below are consumed only by testbenches)
20 package cordic_pkg;
21
22   timeunit      1ns;
23   timeprecision 1ps;
24
25   // -----
26   // Format parameters. `parameter' at package scope behaves like VHDL's
27   // deferred constant: it can be overridden only by a tool switch, not by
28   // an instance, so treat it as a constant.
29   // -----
30   parameter int unsigned DW = 16; // external x/y width
31   parameter int unsigned QF = 14; // external x/y fractional bits
32   parameter int unsigned AW = 16; // external angle width
33   parameter int unsigned QA = 13; // external angle fractional bits
34   parameter int unsigned N = 14; // number of micro-rotations
35   parameter int unsigned GL = 2; // guard bits at the LSB side
36   parameter int unsigned GM = 2; // guard bits at the MSB side
37
38   localparam int unsigned IW = DW + GL + GM; // internal x/y width      (20)
39   localparam int unsigned IQF = QF + GL; // internal x/y frac. bits (16)
40   localparam int unsigned IA = AW + GL; // internal angle width      (18)
41   localparam int unsigned IQA = QA + GL; // internal angle frac.      (15)
42
43   // -----
44   // Types. A `typedef' here plays the role of a VHDL subtype declared in a
45   // package -- but note that SystemVerilog types carry no range checking:
46   // `data_t' is simply "20 bits, interpreted as two's complement".
47   // -----
48   typedef logic signed [DW-1:0] data_t; // external x/y
49   typedef logic signed [AW-1:0] angle_t; // external angle
50   typedef logic signed [IW-1:0] idata_t; // internal x/y
51   typedef logic signed [IA-1:0] iangle_t; // internal angle
52
53   // A packed struct is a *bit vector with named fields*. It can cross a
54   // module port, be assigned as a whole, and be cast to and from a plain
55   // vector -- VHDL records cannot be treated as vectors at all.
56   typedef struct packed {
57     data_t x;
58     data_t y;
59     angle_t theta;
60   } cordic_req_t;
61
62   typedef struct packed {
63     data_t x;

```

```

64     data_t y;
65 } cordic_rsp_t;
66
67 // -----
68 // Elaboration-time constants.
69 //
70 // '$atan' and friends are real-valued system functions. Calling them in a
71 // constant expression is the SystemVerilog equivalent of using
72 // ieee.math_real in a VHDL constant: the value is computed once, by the
73 // tool, and never appears in the netlist.
74 // -----
75 localparam int signed MAX_DATA = (1 << (DW - 1)) - 1;    // 32767
76 localparam int signed MIN_DATA = -(1 << (DW - 1));        // -32768
77
78 // pi/2 and pi in the internal angle format
79 localparam int signed HALF_PI_INT = 51472;    // round(pi/2 * 2**IQA)
80 localparam int signed PI_INT      = 102944;    // round(pi   * 2**IQA)
81 // pi in the *external* angle format, used by the input range check
82 localparam int signed PI_EXT      = 25736;    // round(pi   * 2**QA)
83
84 // The processing gain K = prod sqrt(1 + 2**(-2i)) and its reciprocal, in the
85 // external x/y format. Seed the rotator with XO_UNIT to obtain
86 // (cos theta, sin theta) directly on the output.
87 localparam int signed XO_UNIT = 9949;        // round(2**QF / 1.646760)
88
89 // -----
90 // The arctangent ROM.
91 //
92 // A function that returns an unpacked array, used to initialise a
93 // 'localparam': this is how you build a compile-time table in
94 // SystemVerilog. The whole thing disappears into constants during
95 // elaboration, exactly like a VHDL constant array initialised by a
96 // function.
97 // -----
98 typedef iangle_t atan_rom_t [N];
99
100 function automatic atan_rom_t build_atan_rom();
101     atan_rom_t rom;
102     for (int i = 0; i < int'(N); i++) begin
103         // $atan works on 'real'; $rtoi truncates, so add 0.5 to round.
104         rom[i] = iangle_t'($rtoi($atan(2.0 ** (-i)) * (2.0 ** IQA) + 0.5));
105     end
106     return rom;
107 endfunction
108
109 localparam atan_rom_t ATAN_ROM = build_atan_rom();
110
111 // -----
112 // Narrow the internal datapath back to the external format, with
113 // saturation. Truncation of the GL guard bits is an arithmetic shift
114 // right, which rounds towards minus infinity -- the golden model in
115 // Python does exactly the same, so the two agree bit for bit.
116 // -----
117 // The GL guard bits of 'v' are dropped on purpose.
118 // verilator lint_off UNUSED SIGNAL
119 function automatic data_t sat_narrow(input idata_t v);
120     logic signed [IW-GL-1:0] t;    // DW + GM bits
121     // The low IW-GL bits of an arithmetic shift right by GL are exactly the
122     // part-select v[IW-1:GL]; writing it this way keeps the widths explicit.
123     t = v[IW-1:GL];
124     if      (int'(t) > MAX_DATA) sat_narrow = data_t'(MAX_DATA);
125     else if (int'(t) < MIN_DATA) sat_narrow = data_t'(MIN_DATA);
126     else      sat_narrow = data_t'(t);

```

```
127 endfunction
128 // verilator lint_on UNUSED SIGNAL
129
130 // Convenience for testbenches: fixed point <-> real.
131 function automatic real q_to_real(input int signed v, input int unsigned f);
132     return real'(v) / (2.0 ** f);
133 endfunction
134
135 function automatic int signed real_to_q(input real v, input int unsigned f);
136     return $rtoi(v * (2.0 ** f) + (v >= 0.0 ? 0.5 : -0.5));
137 endfunction
138
139 endpackage : cordic_pkg
140 // verilator lint_on UNUSED PARAM
```

Listing 1.4. `cordic_pkg.sv` — the SystemVerilog package.

```

1 // =====
2 // cordic_rot.sv -- iterative (folded) CORDIC rotator, rotation mode.
3 //
4 // Rotates the vector (x, y) by theta and returns
5 //
6 //      x' = K * ( x*cos(theta) - y*sin(theta) )
7 //      y' = K * ( x*sin(theta) + y*cos(theta) )
8 //
9 // where K ~ 1.6468 is the CORDIC processing gain. Seed the core with
10 // x = cordic_pkg::XO_UNIT, y = 0 to obtain (cos theta, sin theta) directly.
11 //
12 // One micro-rotation is performed per clock cycle, so a transaction takes
13 // N + 2 cycles. Request and response use an independent valid/ready
14 // handshake.
15 //
16 // Accepted input range: |theta| <= pi. A larger angle is flagged by an
17 // immediate assertion; the coarse-rotation stage only removes one quadrant.
18 // =====
19 `timescale 1ns/1ps
20
21 module cordic_rot
22     import cordic_pkg::*;
23     #(
24         // The number of micro-rotations is a module parameter so that the same
25         // source can be elaborated at several precisions -- the equivalent of a
26         // VHDL generic. It defaults to the package value.
27         parameter int unsigned ITER = N
28     ) (
29         input logic   clk_i,
30         input logic   rst_ni,      // asynchronous, active low
31
32         // ---- request channel -----
33         input logic   req_valid_i,
34         output logic   req_ready_o,
35         input data_t   req_x_i,
36         input data_t   req_y_i,
37         input angle_t  req_theta_i,
38
39         // ---- response channel -----
40         output logic   rsp_valid_o,
41         input logic   rsp_ready_i,
42         output data_t   rsp_x_o,
43         output data_t   rsp_y_o
44     );
45
46     localparam int unsigned CNT_W = (ITER > 1) ? $clog2(ITER) : 1;
47
48     // -----
49     // An enumerated type for the state. Unlike a VHDL enumeration, a
50     // SystemVerilog enum has an explicit base type -- here two bits -- which is
51     // what actually goes into the flip-flops. Simulators and waveform viewers
52     // still show the symbolic name.
53     // -----
54     typedef enum logic [1:0] {
55         ST_IDLE   = 2'b00,
56         ST_ROTATE = 2'b01,
57         ST_DONE   = 2'b10
58     } state_e;
59
60     state_e state_q, state_d;
61
62     idata_t x_q, x_d;
63     idata_t y_q, y_d;

```

```

64  iangle_t      z_q, z_d;
65  logic [CNT_W-1:0] iter_q, iter_d;
66
67  // -----
68  // Stage 1: widen and coarse-rotate.
69  //
70  // The micro-rotation series converges only for |z| <= 1.7433 rad, so an
71  // angle outside +-pi/2 is first rotated by a whole quadrant:
72  //   +pi/2 : (x, y) -> (-y,  x)
73  //   -pi/2 : (x, y) -> ( y, -x)
74  // Both are exact -- no arithmetic error is introduced.
75  // -----
76  idata_t x_ext, y_ext, x_load, y_load;
77  iangle_t z_ext, z_load;
78
79  always_comb begin
80      // A cast to a wider *signed* type sign-extends. This is the one place
81      // where SystemVerilog's implicit width rules genuinely help: the cast
82      // says what we mean and the shift then adds the guard bits.
83      x_ext = idata_t'(req_x_i) <<< GL;
84      y_ext = idata_t'(req_y_i) <<< GL;
85      z_ext = iangle_t'(req_theta_i) <<< GL;
86
87      if (z_ext > iangle_t'(HALF_PI_INT)) begin
88          x_load = -y_ext;
89          y_load = x_ext;
90          z_load = z_ext - iangle_t'(HALF_PI_INT);
91      end else if (z_ext < -iangle_t'(HALF_PI_INT)) begin
92          x_load = y_ext;
93          y_load = -x_ext;
94          z_load = z_ext + iangle_t'(HALF_PI_INT);
95      end else begin
96          x_load = x_ext;
97          y_load = y_ext;
98          z_load = z_ext;
99      end
100  end
101
102  // -----
103  // Stage 2: one micro-rotation, purely combinational.
104  //
105  //      d      = sign(z)
106  //      x[i+1] = x[i] - d * (y[i] >>> i)
107  //      y[i+1] = y[i] + d * (x[i] >>> i)
108  //      z[i+1] = z[i] - d * atan(2**--i)
109  //
110  // Note that '>>>' on a signed operand is an arithmetic shift. Getting
111  // this wrong is the single most common bug when porting fixed point code
112  // from VHDL, where shift_right on a 'signed' is arithmetic by definition.
113  // -----
114  idata_t x_sh, y_sh, x_rot, y_rot;
115  iangle_t z_rot, atan_i;
116  logic ccw; // 1 = rotate counter-clockwise
117
118  always_comb begin
119      ccw = ~z_q[IA-1]; // z >= 0
120      x_sh = x_q >>> iter_q;
121      y_sh = y_q >>> iter_q;
122      atan_i = ATAN_ROM[iter_q];
123
124      if (ccw) begin
125          x_rot = x_q - y_sh;
126          y_rot = y_q + x_sh;

```

```

127     z_rot = z_q - atan_i;
128 end else begin
129     x_rot = x_q + y_sh;
130     y_rot = y_q - x_sh;
131     z_rot = z_q + atan_i;
132 end
133 end
134
135 // -----
136 // Stage 3: the control FSM.
137 //
138 // `always_comb' guarantees a complete sensitivity list, and the default
139 // assignments at the top of the block guarantee that every signal is
140 // written on every path -- which is how you avoid an inferred latch.
141 // `unique case' asks the simulator to check at run time that exactly one
142 // branch matches; there is no VHDL equivalent short of writing the
143 // assertion yourself.
144 // -----
145 always_comb begin
146     state_d      = state_q;
147     x_d          = x_q;
148     y_d          = y_q;
149     z_d          = z_q;
150     iter_d       = iter_q;
151     req_ready_o  = 1'b0;
152     rsp_valid_o  = 1'b0;
153
154     unique case (state_q)
155         ST_IDLE: begin
156             req_ready_o = 1'b1;
157             if (req_valid_i) begin
158                 x_d      = x_load;
159                 y_d      = y_load;
160                 z_d      = z_load;
161                 iter_d   = '0;
162                 state_d  = ST_ROTATE;
163             end
164         end
165
166         ST_ROTATE: begin
167             x_d = x_rot;
168             y_d = y_rot;
169             z_d = z_rot;
170             if (iter_q == CNT_W'(ITER - 1)) begin
171                 state_d = ST_DONE;
172             end else begin
173                 iter_d = iter_q + CNT_W'(1);
174             end
175         end
176
177         ST_DONE: begin
178             rsp_valid_o = 1'b1;
179             if (rsp_ready_i) state_d = ST_IDLE;
180         end
181
182         default: state_d = ST_IDLE;
183     endcase
184 end
185
186 // -----
187 // Stage 4: the state registers.
188 //
189 // One `always_ff' per clock domain, non-blocking assignments only. The

```

```

190 // asynchronous reset appears in the sensitivity list exactly as it does in
191 // a VHDL process.
192 // -----
193 always_ff @(posedge clk_i or negedge rst_ni) begin
194     if (!rst_ni) begin
195         state_q <= ST_IDLE;
196         x_q      <= '0;
197         y_q      <= '0;
198         z_q      <= '0;
199         iter_q   <= '0;
200     end else begin
201         state_q <= state_d;
202         x_q     <= x_d;
203         y_q     <= y_d;
204         z_q     <= z_d;
205         iter_q  <= iter_d;
206     end
207 end
208
209 // -----
210 // Outputs. Saturating narrow-down lives in the package so that the
211 // testbench can call exactly the same function.
212 // -----
213 assign rsp_x_o = sat_narrow(x_q);
214 assign rsp_y_o = sat_narrow(y_q);
215
216 // -----
217 // Built-in checks. These are part of the *design*, not of a testbench:
218 // they travel with the module and fire in every simulation that
219 // instantiates it. Synthesis tools ignore them.
220 // -----
221 `ifndef SYNTHESIS
222 // Immediate assertion: the accepted input range.
223 always_ff @(posedge clk_i) begin
224     if (rst_ni && req_valid_i && req_ready_o) begin
225         assert (req_theta_i <= angle_t'(PI_EXT) &&
226             req_theta_i >= -angle_t'(PI_EXT))
227             else $error("cordic_rot: |theta| > pi (theta = %0d)",
228                 $signed(req_theta_i));
229     end
230 end
231
232 // Concurrent assertion: a request must stay stable while it is stalled.
233 property p_req_stable;
234     @(posedge clk_i) disable iff (!rst_ni)
235         (req_valid_i && !req_ready_o) | =>
236             $stable(req_x_i) && $stable(req_y_i) && $stable(req_theta_i);
237 endproperty
238 a_req_stable : assert property (p_req_stable)
239     else $error("cordic_rot: request payload changed while stalled");
240 `endif
241
242 endmodule : cordic_rot

```

Listing 1.5. `cordic_rot.sv` — the SystemVerilog implementation.

1.7 The Two Implementations Compared

Table 1.3 counts the lines. Both files are commented at the same density, so the ratio is meaningful.

Table 1.3. Source size. Comments and blank lines are included, because the two files were written to the same commenting standard.

	VHDL	SystemVerilog
package	148	140
implementation	253	242
testbench	290	689 (interface + package + top)
design total	401	382

The design is a wash: 401 lines against 382, five per cent. That is worth saying plainly, because the folklore holds that SystemVerilog is dramatically terser. For *structural* RTL it is not. The savings that folklore is remembering come from interfaces, from packed structs, and above all from testbenches, and this design uses none of the first two.

The testbench is the opposite: 290 lines against 689, and the SystemVerilog version does more. That is the real comparison, and Section 1.8 and Section 1.9 put the two side by side.

Four places where the languages genuinely differ for this design:

The type system did work for the VHDL. Three width errors were caught by the analyser while this design was being written, before anything was simulated. The equivalent SystemVerilog errors surfaced as `WIDTHEXPAND` and `WIDTHTRUNC` warnings from `verilator -lint-only -Wall`, which is a perfectly good substitute *provided you run it*, and which no student runs by default. The mitigation is a `make lint` target that fails the build.

The constant table is a draw with a portability asterisk. Both languages compute it at elaboration from a transcendental function. VHDL's version works in every tool that implements the standard; SystemVerilog's does not (see the trap in Section 1.6).

The enumeration encoding favours VHDL for synthesis and SystemVerilog for debugging. A VHDL state type lets the tool pick the encoding; a SystemVerilog one tells you what is in the flip-flops. Which you want depends on whether you are closing timing or reading a waveform.

Assertions favour SystemVerilog decisively. One temporal property is four readable lines in SystemVerilog and twenty unreadable ones in VHDL. Multiply that by the ten protocol properties a real bus interface has.

And one thing this design should have used

`ieee.fixed_pkg` was available for the VHDL and was not used, so that the two implementations would be structurally comparable. A production VHDL design of this kind should use it: it would have made Table 1.2 machine-checkable instead of documentation.

1.8 The VHDL Testbench

Listing 1.6 is a self-checking VHDL testbench, written the way a VHDL designer writes one: a stimulus process, a checking process, procedures instead of classes, and a golden model built from `ieee.math_real`. It is the baseline the rest of the chapter is measured against.

1.8.1 The parts, and what each one costs

The generic is the VHDL plusarg.

```

1 entity tb_cordic_rot is
2   generic (
3     G_N_RANDOM : natural := 300;
4     G_TOL_LSB  : natural := 12;
5     G_SEED1    : positive := 7;
6     G_SEED2    : positive := 13
7   );
8 end entity tb_cordic_rot;
```

Elaborate it with `vsim -gG_N_RANDOM=1000 tb_cordic_rot` and you change the run without touching the source. Unlike a SystemVerilog plusarg, a generic is fixed at elaboration rather than read at run time, so changing it means elaborating again, not merely restarting.

The clock, with a stop condition.

```

1 clk <= not clk after CLK_PERIOD / 2 when not halt else '0';
```

A VHDL simulation traditionally ends by running out of events, so the clock has to be stoppable. `std.env.finish` (VHDL-2008) is the modern alternative and the testbench uses both: it sets `halt` and then calls `finish`.

Driving on the falling edge. The `send` procedure asserts `req_valid` and then waits for falling edges:

```

1 procedure send (x, y, theta : integer) is
2 begin
3   req_x      <= to_signed(x, DW);
4   req_theta <= to_signed(theta, AW);
5   req_valid <= '1';
6   loop
7     wait until falling_edge(clk);
8     exit when req_ready = '1';
9   end loop;
10  wait until rising_edge(clk);
11  wait until falling_edge(clk);
12  req_valid <= '0';
13 end procedure send;
```

This is the convention that keeps the testbench out of a race with the DUT, and it works. It is also a lie about the hardware: no real driver changes its outputs half a cycle before the clock. A SystemVerilog clocking block expresses the truth — drive 1 ns after the edge, sample the value that existed just before it — and does not need the convention.

No constraint solver.

```

1 impure function rand_int (lo, hi : integer) return integer is
2 begin
3   uniform(seed1, seed2, r);
4   return lo + integer(floor(r * real(hi - lo + 1)));
5 end function rand_int;
```

uniform from `ieee.math_real` is the whole of VHDL's random number support. A uniform distribution over an interval is six lines; a distribution weighted toward the quadrant boundaries is however many lines you care to write. In SystemVerilog the same thing is a `dist` clause inside a `constraint` block. This is a real productivity difference, and it is the one OSVVM exists to close.

The structural point: monitor and scoreboard cannot be separated.

```

1  p_check : process is
2      variable pend    : req_array_t(0 to Q_DEPTH - 1);
3      variable p_head  : natural := 0;
4      variable p_tail  : natural := 0;
5  begin
6      ...
7      if req_valid = '1' and req_ready = '1' then          -- monitor
8          pend(p_tail) := (x => to_integer(req_x), ...);
9          p_tail := (p_tail + 1) mod Q_DEPTH;
10     end if;
11     if rsp_valid = '1' and rsp_ready = '1' then          -- scoreboard
12         cur := pend(p_head);
13     ...

```

The observation of the request channel and the checking of the response channel are in one process because they have to share the pending queue, and a VHDL process variable is the only cheap way to share mutable state. There is no mailbox to put a transaction into and no dynamic object to put in it.

The consequence is not that the testbench is worse — it works, and it found bugs — but that it does not decompose. You cannot lift the monitor out and reuse it against a gate-level netlist, or run two of them, or subscribe a coverage collector to the same stream, without rewriting.

△ Trap: Sharing a queue through a signal does not work

The first version of this testbench logged each accepted request into a `signal` array indexed by a counter, for the checking process to read. Every entry read back as zero. A signal assignment inside a process takes effect after the process suspends, the index is itself a signal, and the two delta-cycle behaviours interact in a way that is correct by the standard and useless in practice. The fix was to make the checking process observe the request channel itself, which is what a SystemVerilog monitor does anyway.

1.8.2 Running it

Three commands: make a library, analyse the three files in dependency order, elaborate and run the top.

```

vlib work
vcom -2008 cordic_pkg.vhd cordic_rot.vhd tb_cordic_rot.vhd
vsim -c -g N_RANDOM=300 tb_cordic_rot -do "run -all; quit"

```

Everything the testbench has to say about itself comes from `report` statements, so the summary reads the same whatever tool executed it. Stripped of the per-simulator prefix that decorates each line, the run prints:

```

=== cordic_rot testbench (VHDL) ===
ITER=14  random=300
-----
scoreboard: 308 transactions checked, 0 failed
worst error: 2.5458225901211335 LSB (tolerance 12)

```

```
-----
*** TEST PASSED ***
```

308 transactions rather than 300, because the directed list runs first and the random phase is counted on top of it. The worst error, 2.55 LSB, is the number Section 1.3 predicted from the error budget.

Before any of that, at time zero, the run also emits a handful of warnings from `numeric_std` itself, of the form `NUMERIC_STD.">": metavalue detected, returning FALSE`. They are worth understanding rather than silencing by reflex.

Those metavalue warnings are VHDL telling you something

At time zero, before the first assignment has propagated, the DUT's inputs are 'U'. The comparison `z_ext > HALF_PI_S` on a vector containing 'U' cannot produce a meaningful answer, so `numeric_std` says so and returns `FALSE`. The message comes from the IEEE library, not from the simulator, so every compliant tool reports it. Verilog has no 'U': an uninitialised variable is `X`, and `X > 5` quietly evaluates to `X` and then to `false` in an `if`, with no warning at all. VHDL is being more informative, not more broken.

For a regression, where the noise buries the result, put

```
set NumericStdNoWarnings 1
```

in the `do` file before `run -all`. Do not set it while you are debugging a reset: those warnings are exactly the evidence that something is still 'U' when you thought it was driven.

```

1  -- =====
2  --  tb_cordic_rot.vhd -- a self-checking VHDL testbench for the CORDIC.
3  --
4  --  This is deliberately written the way a VHDL designer writes a testbench:
5  --  one stimulus process, one checking process, procedures instead of classes,
6  --  and a golden model built from ieee.math_real.  Read it next to
7  --  tb_cordic_pkg.sv and tb_cordic_top.sv to see exactly what the class-based
8  --  structure buys and what it costs.
9  --
10 --  Run with:
11 --      vlib work
12 --      vcom -2008 cordic_pkg.vhd cordic_rot.vhd tb_cordic_rot.vhd
13 --      vsim -c -gG_N_RANDOM=300 -do "run -all; quit" tb_cordic_rot
14 -- =====
15 library ieee;
16 use ieee.std_logic_1164.all;
17 use ieee.numeric_std.all;
18 use ieee.math_real.all;
19
20 library std;
21 use std.textio.all;
22 use std.env.finish;
23
24 library work;
25 use work.cordic_pkg.all;
26
27 entity tb_cordic_rot is
28 generic (
29     -- A generic on a testbench is the VHDL answer to a plusarg: it lets you
30     -- change the run without editing the source.  Unlike a plusarg it is
31     -- fixed at elaboration: -gG_N_RANDOM=... is given to the elaboration
32     -- step, not to the running simulation.
33     G_N_RANDOM : natural := 300;
34     G_TOL_LSB   : natural := 12;
35     G_SEED1     : positive := 7;
36     G_SEED2     : positive := 13
37 );
38 end entity tb_cordic_rot;
39
40
41 architecture sim of tb_cordic_rot is
42
43     -- The CORDIC processing gain,  $K = \text{product } \sqrt{1 + 2^{*-2i}}$ .
44     constant K_GAIN : real := 1.6467602581210656;
45
46     constant CLK_PERIOD : time := 10 ns;
47
48     signal clk      : std_logic := '0';
49     signal rst_n    : std_logic := '0';
50     signal halt     : boolean   := false;
51
52     signal req_valid : std_logic := '0';
53     signal req_ready : std_logic;
54     signal req_x     : data_t    := (others => '0');
55     signal req_y     : data_t    := (others => '0');
56     signal req_theta : angle_t   := (others => '0');
57
58     signal rsp_valid : std_logic;
59     signal rsp_ready : std_logic := '0';
60     signal rsp_x     : data_t;
61     signal rsp_y     : data_t;
62
63     -- The request record.  A VHDL testbench has no mailbox and no dynamic

```

```

64  -- object, so a transaction is a record and the queue is an array of them
65  -- held in a process variable. This is the single biggest structural
66  -- difference from the SystemVerilog version: there, the monitor puts a
67  -- class handle into a mailbox and the scoreboard gets it; here, the
68  -- monitor and the scoreboard must live in the same process so that they
69  -- can share a variable.
70  type req_rec_t is record
71      x      : integer;
72      y      : integer;
73      theta : integer;
74  end record req_rec_t;
75
76  type req_array_t is array (natural range <>) of req_rec_t;
77
78  constant N_DIRECTED : natural := 8;
79  constant N_TOTAL    : natural := N_DIRECTED + G_N_RANDOM;
80  constant Q_DEPTH    : natural := 8;    -- outstanding requests
81
82  signal n_checked : natural := 0;
83  signal n_failed  : natural := 0;
84  signal worst_err : real     := 0.0;
85
86  begin
87
88      -----
89      -- Clock. The classic VHDL idiom. `halt' stops it so that the simulation
90      -- ends by running out of events, which is how a VHDL testbench normally
91      -- terminates; std.env.finish is the 2008 alternative and is used below.
92      -----
93      clk <= not clk after CLK_PERIOD / 2 when not halt else '0';
94
95      -----
96      -- Device under test
97      -----
98      dut : entity work.cordic_rot(rtl)
99          generic map (ITER => N)
100          port map (
101              clk_i      => clk,
102              rst_ni     => rst_n,
103              req_valid_i => req_valid,
104              req_ready_o => req_ready,
105              req_x_i    => req_x,
106              req_y_i    => req_y,
107              req_theta_i => req_theta,
108              rsp_valid_o => rsp_valid,
109              rsp_ready_i => rsp_ready,
110              rsp_x_o    => rsp_x,
111              rsp_y_o    => rsp_y
112          );
113
114      -----
115      -- Stimulus.
116      --
117      -- Everything the SystemVerilog driver and generator do together is here in
118      -- one process. Note the two habits that VHDL testbenches rely on and
119      -- SystemVerilog replaces with language features:
120      --   * driving on the *falling* edge to avoid a race with the DUT, where
121      --     SystemVerilog uses a clocking block;
122      --   * a `procedure' with signal parameters, where SystemVerilog uses a
123      --     class method and a virtual interface.
124      -----
125      p_stim : process is
126

```

```

127 variable seed1 : positive := G_SEED1;
128 variable seed2 : positive := G_SEED2;
129 variable r      : real;
130
131 -- uniform() is the standard pseudo-random generator. There is no
132 -- constraint solver: a distribution is something you write yourself.
133 impure function rand_int (lo, hi : integer) return integer is
134 begin
135     uniform(seed1, seed2, r);
136     return lo + integer(floor(r * real(hi - lo + 1)));
137 end function rand_int;
138
139 procedure send (x, y, theta : integer) is
140 begin
141     req_x    <= to_signed(x,    DW);
142     req_y    <= to_signed(y,    DW);
143     req_theta <= to_signed(theta, AW);
144     req_valid <= '1';
145     -- Hold the request until the DUT takes it.
146     loop
147         wait until falling_edge(clk);
148         exit when req_ready = '1';
149     end loop;
150     -- The checker snoops the request channel itself, so there is nothing
151     -- to log here.
152     wait until rising_edge(clk);
153     wait until falling_edge(clk);
154     req_valid <= '0';
155 end procedure send;
156
157 begin
158     report "=== cordic_rot testbench (VHDL) ===";
159     report "ITER=" & integer'image(N) &
160           " random=" & integer'image(G_N_RANDOM);
161
162     -- reset
163     rst_n    <= '0';
164     rsp_ready <= '0';
165     for i in 0 to 4 loop
166         wait until falling_edge(clk);
167     end loop;
168     rst_n    <= '1';
169     rsp_ready <= '1';
170     wait until falling_edge(clk);
171
172     -- ---- directed corners -----
173     send(XO_UNIT, 0, 0);
174     send(XO_UNIT, 0, PI_EXT);
175     send(XO_UNIT, 0, -PI_EXT);
176     send(XO_UNIT, 0, PI_EXT / 2);
177     send(XO_UNIT, 0, -PI_EXT / 2);
178     send(XO_UNIT, 0, PI_EXT / 4);
179     send(0, XO_UNIT, PI_EXT / 2);
180     send(0, 0, PI_EXT / 3);
181
182     -- ---- random -----
183     for i in 0 to G_N_RANDOM - 1 loop
184         send(rand_int(-6000, 6000),
185             rand_int(-6000, 6000),
186             rand_int(-PI_EXT, PI_EXT));
187     end loop;
188
189     wait;

```

```

190 end process p_stim;
191
192 -----
193 -- Checker.
194 --
195 -- The equivalent of the SystemVerilog monitor and scoreboard, fused into
196 -- one process because there is no cheap way to pass a transaction between
197 -- two processes in VHDL.
198 -----
199 p_check : process is
200     -- The pending queue: requests accepted whose result has not come back.
201     variable pend : req_array_t(0 to Q_DEPTH - 1);
202     variable p_head : natural := 0;
203     variable p_tail : natural := 0;
204     variable cur : req_rec_t;
205     variable idx : natural := 0;
206     variable th, xr, yr, x_ref, y_ref, ex, ey : real;
207     variable line_v : line;
208 begin
209     wait until rising_edge(clk) and rst_n = '1';
210
211     while idx < N_TOTAL loop
212         wait until rising_edge(clk);
213
214         ----- monitor the request channel -----
215         if req_valid = '1' and req_ready = '1' then
216             pend(p_tail) := (x => to_integer(req_x),
217                             y => to_integer(req_y),
218                             theta => to_integer(req_theta));
219             p_tail := (p_tail + 1) mod Q_DEPTH;
220         end if;
221
222         ----- monitor the response channel and check -----
223         if rsp_valid = '1' and rsp_ready = '1' then
224             cur := pend(p_head);
225             p_head := (p_head + 1) mod Q_DEPTH;
226
227             -- the specification, not the implementation
228             th := q_to_real(cur.theta, QA);
229             xr := q_to_real(cur.x, QF);
230             yr := q_to_real(cur.y, QF);
231             x_ref := K_GAIN * (xr * cos(th) - yr * sin(th));
232             y_ref := K_GAIN * (xr * sin(th) + yr * cos(th));
233
234             ex := abs(q_to_real(to_integer(rsp_x), QF) - x_ref) * 2.0 ** QF;
235             ey := abs(q_to_real(to_integer(rsp_y), QF) - y_ref) * 2.0 ** QF;
236
237             if ex > worst_err then worst_err <= ex; end if;
238             if ey > worst_err then worst_err <= ey; end if;
239
240             if ex > real(G_TOL_LSB) or ey > real(G_TOL_LSB) then
241                 n_failed <= n_failed + 1;
242                 write(line_v, string("MISMATCH #"));
243                 write(line_v, idx);
244                 write(line_v, string(" theta="));
245                 write(line_v, cur.theta);
246                 write(line_v, string(" got x="));
247                 write(line_v, to_integer(rsp_x));
248                 write(line_v, string(" y="));
249                 write(line_v, to_integer(rsp_y));
250                 write(line_v, string(" want x="));
251                 write(line_v, integer(round(x_ref * 2.0 ** QF)));
252                 write(line_v, string(" y="));

```

```

253     write(line_v, integer(round(y_ref * 2.0 ** QF)));
254     writeline(output, line_v);
255     end if;
256
257     idx      := idx + 1;
258     n_checked <= idx;
259     end if;
260 end loop;
261
262 -- ---- report -----
263 wait for 1 ns;
264 report "-----";
265 report "scoreboard: " & integer'image(n_checked) &
266       " transactions checked, " & integer'image(n_failed) & " failed";
267 report "worst error: " & real'image(worst_err) &
268       " LSB (tolerance " & integer'image(G_TOL_LSB) & " LSB)";
269 report "-----";
270
271 assert n_failed = 0
272     report "*** TEST FAILED ***" severity failure;
273 report "*** TEST PASSED ***";
274
275 halt <= true;
276 finish;
277 end process p_check;
278
279 -- -----
280 -- Watchdog. Without one a deadlocked DUT gives you a simulation that runs
281 -- until you notice.
282 -- -----
283 p_watchdog : process is
284 begin
285     wait for 200 us;
286     report "global timeout: the testbench did not finish"
287         severity failure;
288 end process p_watchdog;
289
290 end architecture sim;

```

Listing 1.6. `tb_cordic_rot.vhd` — the VHDL testbench, complete.

1.9 The SystemVerilog Testbench

Three files, 689 lines, and every structural idea from the *Verification Cookbook* in working form: an interface with clocking blocks, six classes, mailboxes between them, a directed phase followed by a random one, and an end of test that drains rather than guesses.

1.9.1 The interface

Listing 1.7 is the pin bundle. The part that matters is the clocking block:

```

1  clocking drv_cb @(posedge clk);
2      default input #1step output #1ns;
3      output req_valid, req_x, req_y, req_theta;
4      output rsp_ready;
5      input  req_ready, rsp_valid, rsp_x, rsp_y;
6  endclocking

```

input #1step samples in the Preponed region: the value that existed immediately before the clock edge, before any non-blocking update. output #1ns drives one nanosecond after the edge. Together they remove the entire class of bug that the VHDL testbench avoids by driving on the falling edge, and they model what a real driver does.

1.9.2 The six classes

Listing 1.8 is the testbench package. The shape is exactly the *Verification Cookbook*:

- cordic_txn — the transaction, with rand fields, constraints (including a dist that biases toward the quadrant boundaries), and convert2string;
- cordic_drv — the driver, which knows the handshake and nothing else, and runs two threads: one feeding requests, one applying random back-pressure to the response channel;
- cordic_mon — the monitor, strictly passive, holding a queue of accepted-but-unanswered requests;
- cordic_sb — the scoreboard and its specification-level reference model;
- cordic_cov — the coverage collector, with a covergroup that crosses the angle quadrant against the sign of x_0 ;
- cordic_env — construction, connection, start-up.

The reference model is worth quoting because of what it does *not* say:

```

1  function automatic void predict(input cordic_txn t,
2                                  output real x_ref, output real y_ref);
3      real th = q_to_real(t.theta, QA);
4      real xr = q_to_real(t.x0,   QF);
5      real yr = q_to_real(t.y0,   QF);
6      x_ref = K_GAIN * (xr * $cos(th) - yr * $sin(th));
7      y_ref = K_GAIN * (xr * $sin(th) + yr * $cos(th));
8  endfunction

```

No micro-rotations, no guard bits, no quadrant folding. It states what the block is supposed to compute. Every implementation detail is something it can therefore catch.

1.9.3 The portability shim

At the top of `tb_cordic_pkg.sv` sit four macros, and they need explaining because they are the one piece of the design that exists for a tool rather than for the hardware.

```

1  `ifdef VERILATOR
2  `define CB_DRV      vif
3  `define CB_MON      vif
4  `define EDGE_DRV    @(posedge vif.clk)
5  `define EDGE_MON    @(posedge vif.clk)
6  `else
7  `define CB_DRV      vif.drv_cb
8  `define CB_MON      vif.mon_cb
9  `define EDGE_DRV    @(vif.drv_cb)
10 `define EDGE_MON    @(vif.mon_cb)
11 `endif

```

Verilator 5.020 has two problems with clocking blocks reached through a virtual interface, and the second one is nasty.

The first is honest: a `modport` that exports a clocking block is rejected with `Unsupported: Modport clocking`. You see the message, you work around it.

The second is not. A *non-blocking* assignment through a virtual interface to a signal that is a clocking-block output is silently dropped. The write never happens; nothing is reported. This was not assumed — it was reduced to a ten-line test case and confirmed:

```

1 interface bif (input logic clk);
2     logic a, c;
3     `ifndef NOCB
4         clocking cb @(posedge clk); output a; endclocking
5     `endif
6 endinterface
7 // ... a class with `virtual bif v` doing:
8 //     v.a <= 1'b1;    // in the clocking block
9 //     v.c <= 1'b1;    // not in any clocking block

```

=== WITH clocking block ===	=== WITHOUT clocking block ===
[5] a=0 c=0	[5] a=0 c=0
[15] a=0 c=1	[15] a=1 c=1
[25] a=0 c=1	[25] a=1 c=1

`c` takes the assignment; `a` does not, purely because `a` is named in a clocking block. The symptom in the real testbench was that `req_valid` never went high, the DUT never left `IDLE`, and the run timed out after 42 600 cycles with zero transactions completed — with no error message pointing anywhere near the cause.

△ Trap: A silently dropped write is the worst failure mode there is

An unsupported construct that produces an error message costs you ten minutes. One that produces wrong behaviour with no message costs you an afternoon, and it teaches you to distrust the tool rather than the code. This is the strongest argument in the book for having a second simulator: the same sources under `QuestaSim` and under `Verilator` would have located this in one run.

The shim keeps one set of sources runnable in both worlds. Under any IEEE 1800 simulator the macros expand to the clocking block, with its sampling and drive skew; under `Verilator` they expand to the raw signals plus an explicit clock edge, which loses the skew and keeps the test.

1.9.4 Ending the test

```

1 task automatic wait_drain(int unsigned expected, int unsigned timeout_cyc);
2   int unsigned waited = 0;
3   while (sb.n_checked < expected && waited < timeout_cyc) begin
4     `EDGE_MON;
5     waited++;
6   end
7   if (sb.n_checked < expected)
8     $fatal(1, "timeout: only %0d of %0d transactions completed",
9           sb.n_checked, expected);
10  endtask

```

Count what went in, wait until that many have been checked, fail if they never arrive. This is precisely what UVM's objection mechanism automates, and writing it once by hand is the fastest way to understand why the mechanism exists (the *Verification Cookbook*).

1.9.5 Running it

```

$ make sv
=== cordic_rot testbench =====
ITER=14  DW=16.Q14  AW=16.Q13  random=300
-----
scoreboard: 308 transactions checked, 0 failed
worst error: 2.82 LSB (tolerance 12 LSB)
-----
*** TEST PASSED ***
functional coverage: not collected (Verilator build)

```

Note the last line. The covergroups are compiled out under Verilator, and so are the constraints: `cordic_txn::do_randomize()` falls back to `$urandom_range`. Under QuestaSim the same three files run with constrained random stimulus and a coverage number at the end. That is what the *Simulation Cookbook* means when it says Verilator implements a subset: the subset is large enough to run this testbench and small enough that two of its most valuable features go quiet.

```

1 // =====
2 // cordic_if.sv -- the testbench view of the CORDIC pins.
3 //
4 // An interface is the single construct that VHDL has no answer to. It
5 // bundles the wires, the *timing* with which the testbench uses them
6 // (clocking blocks) and the direction each participant sees (modports).
7 //
8 // In VHDL the same job needs: a record type in a package, a second record
9 // for the opposite direction, hand-written delays in every driver, and a
10 // block of port map lines in every instantiation.
11 // =====
12 `timescale 1ns/1ps
13
14 interface cordic_if (input logic clk);
15
16     import cordic_pkg::*;
17
18     logic    rst_n;
19
20     logic    req_valid;
21     logic    req_ready;
22     data_t   req_x;
23     data_t   req_y;
24     angle_t  req_theta;
25
26     logic    rsp_valid;
27     logic    rsp_ready;
28     data_t   rsp_x;
29     data_t   rsp_y;
30
31     // -----
32     // Clocking block for the driver.
33     //
34     // `default input #1step' samples in the *Preponed* region, i.e. the value
35     // that existed just before the clock edge -- no race with the DUT's own
36     // non-blocking updates. `output #1ns' drives 1 ns after the edge, which
37     // is what a real ATE or a real board would do.
38     //
39     // This is the construct that removes the whole class of "my testbench
40     // reads the new value instead of the old one" bugs that VHDL solves by
41     // convention (drive on the falling edge) rather than by language.
42     // -----
43     // Note: version 5.020 of Verilator silently DROPS a non-blocking assignment
44     // made through a virtual interface to a signal that is a clocking-block
45     // output: the write simply never happens, with no warning. The clocking
46     // blocks are therefore compiled out of that build; see Chapter 8.
47     `ifndef VERILATOR
48         clocking drv_cb @(posedge clk);
49         default input #1step output #1ns;
50         output req_valid, req_x, req_y, req_theta;
51         output rsp_ready;
52         input  req_ready, rsp_valid, rsp_x, rsp_y;
53     endclocking
54
55     // Passive: everything is an input.
56     clocking mon_cb @(posedge clk);
57     default input #1step;
58     input req_valid, req_ready, req_x, req_y, req_theta;
59     input rsp_valid, rsp_ready, rsp_x, rsp_y;
60     endclocking
61 `endif
62
63     // -----

```

```

64 // Modports name the roles. `drv' and `mon' hand out a clocking block, so
65 // a class that uses them physically cannot drive a signal at the wrong
66 // time.
67 // -----
68 modport dut (
69     input  clk, rst_n,
70     input  req_valid, req_x, req_y, req_theta,
71     output req_ready,
72     output rsp_valid, rsp_x, rsp_y,
73     input  rsp_ready
74 );
75
76 // A modport that hands out a clocking block is legal IEEE 1800 but is not
77 // yet supported by Verilator 5.020, so it is guarded. The testbench
78 // classes reach the clocking blocks directly through the virtual
79 // interface, which works everywhere.
80 `ifndef VERILATOR
81     modport drv (clocking drv_cb, output rst_n, input clk);
82     modport mon (clocking mon_cb, input rst_n, input clk);
83 `endif
84
85 endinterface : cordic_if

```

Listing 1.7. `cordic_if.sv` — the interface, its clocking blocks and its modports.

```

1 // =====
2 // tb_cordic_pkg.sv -- a layered, class-based testbench without UVM.
3 //
4 // Everything UVM gives you is here in about 350 lines: a transaction, a
5 // stimulus generator, a driver, a passive monitor, a scoreboard with a
6 // reference model, a coverage collector and an environment that wires them
7 // together. Nothing is imported from a library; the only reason UVM exists
8 // is that it standardises *these* names and adds configuration, factory
9 // overrides and phasing on top.
10 //
11 // Read this file before Chapter 7. UVM will then look like what it is: the
12 // same six boxes with a base class in front of each one.
13 // =====
14 `timescale 1ns/1ps
15
16 // -----
17 // Portability shim.
18 //
19 // The natural way to reach a clocking block from a class is
20 // `vif.drv_cb.req_valid <= v'. Verilator 5.020 cannot do that. A modport
21 // that exports a clocking block is rejected outright, and -- worse -- a
22 // non-blocking assignment through a virtual interface to a clocking-block
23 // output is silently DROPPED: the write never happens and no warning is
24 // issued. See Chapter 8 for the ten-line test case that demonstrates it.
25 //
26 // Rather than write the testbench twice, the four access points are hidden
27 // behind four macros. In any IEEE 1800 simulator they expand to the clocking
28 // block, with its Preponed sampling and its output skew. Under Verilator
29 // they expand to the raw interface signals plus an explicit clock edge, which
30 // loses the skew but keeps the testbench runnable.
31 //
32 // This is what portability actually costs, and it is worth seeing once.
33 // -----
34 `ifndef VERILATOR
35 `define CB_DRV      vif
36 `define CB_MON      vif
37 `define EDGE_DRV    @(posedge vif.clk)
38 `define EDGE_MON    @(posedge vif.clk)
39 `else
40 `define CB_DRV      vif.drv_cb
41 `define CB_MON      vif.mon_cb
42 `define EDGE_DRV    @(vif.drv_cb)
43 `define EDGE_MON    @(vif.mon_cb)
44 `endif
45
46 package tb_cordic_pkg;
47
48 import cordic_pkg::*;
49
50 // The CORDIC processing gain,  $K = \prod_{i=0}^{N-1} \sqrt{1 + 2^{-2i}}$ .
51 localparam real K_GAIN = 1.6467602581210656;
52
53 // Comparison tolerance, in LSBs of the output format. The theoretical
54 // bound for  $N = 14$  micro-rotations plus the guard-bit truncation is a
55 // little under 5 LSB; 12 leaves margin without hiding a real bug.
56 localparam int TOL_LSB = 12;
57
58 // =====
59 // 1. The transaction
60 //
61 // A transaction is a *value*, not a component: it is created, passed
62 // around and thrown away. In UVM it would extend uvm_sequence_item; the
63 // only thing that adds is automatic copy/compare/print.

```

```

64 // =====
65 class cordic_txn;
66
67 // ---- stimulus -----
68 rand data_t x0;
69 rand data_t y0;
70 rand angle_t theta;
71
72 // ---- observed result, filled in by the monitor -----
73 data_t x;
74 data_t y;
75
76 // ---- bookkeeping -----
77 int unsigned id;
78 static int unsigned next_id = 0;
79
80 `ifndef VERILATOR
81 // Constrained random. `inside' with a range is the SystemVerilog way of
82 // saying "pick uniformly in this interval"; there is no VHDL equivalent
83 // short of writing the generator yourself.
84 constraint c_theta {
85     theta inside {[-PI_EXT : PI_EXT]};
86 }
87 // Keep |(x0, y0)| * K comfortably inside the output range so that the
88 // saturating output stage is exercised only by the directed tests.
89 constraint c_magnitude {
90     x0 inside {[-6000 : 6000]};
91     y0 inside {[-6000 : 6000]};
92 }
93 // Bias the distribution towards the interesting angles: the quadrant
94 // boundaries, zero, and the extremes.
95 constraint c_corners {
96     theta dist {
97         0                :/ 1,
98         PI_EXT           :/ 1,
99         -PI_EXT          :/ 1,
100         PI_EXT / 2       :/ 1,
101         -PI_EXT / 2      :/ 1,
102         [-PI_EXT:PI_EXT] :/ 20
103     };
104 }
105 `endif
106
107 function new();
108     id = next_id++;
109 endfunction
110
111 // The Verilator build has no constraint solver, so the same class also
112 // carries a hand-written fallback. Chapter 8 explains why this is a
113 // reasonable trade-off rather than a defect.
114 function void randomize_fallback();
115     theta = angle_t'($urandom_range(2 * PI_EXT) - PI_EXT);
116     x0 = data_t'($urandom_range(12000) - 6000);
117     y0 = data_t'($urandom_range(12000) - 6000);
118 endfunction
119
120 function void do_randomize();
121 `ifdef VERILATOR
122     randomize_fallback();
123 `else
124     if (!this.randomize()) $fatal(1, "randomization failed");
125 `endif
126 endfunction

```

```

127
128     function string convert2string();
129         return $sformatf(
130             "txn#%0d x0=%0d (%.4f) y0=%0d (%.4f) theta=%0d (%.4f rad)",
131             id, x0, q_to_real(x0, QF), y0, q_to_real(y0, QF),
132             theta, q_to_real(theta, QA));
133     endfunction
134
135 endclass : cordic_txn
136
137
138 // =====
139 // 2. The driver
140 //
141 // It knows the pin-level protocol and nothing else. Every access to the
142 // DUT goes through the clocking block, so the driver cannot accidentally
143 // create a race with the DUT's own non-blocking updates.
144 // =====
145 class cordic_drv;
146
147     virtual cordic_if vif;
148     mailbox #(cordic_txn) req_mbx;          // from the generator
149     int unsigned stall_pct = 25;           // random back-pressure on the output
150
151     function new(virtual cordic_if vif, mailbox #(cordic_txn) req_mbx);
152         this.vif = vif;
153         this.req_mbx = req_mbx;
154     endfunction
155
156     task automatic reset();
157         vif.rst_n      <= 1'b0;
158         `CB_DRV.req_valid <= 1'b0;
159         `CB_DRV.req_x    <= '0;
160         `CB_DRV.req_y    <= '0;
161         `CB_DRV.req_theta <= '0;
162         `CB_DRV.rsp_ready <= 1'b0;
163         for (int i = 0; i < 5; i++) `EDGE_DRV;
164         vif.rst_n <= 1'b1;
165         `EDGE_DRV;
166     endtask
167
168     // Drive one request and hold it until the DUT accepts it.
169     task automatic send(cordic_txn t);
170         `CB_DRV.req_valid <= 1'b1;
171         `CB_DRV.req_x     <= t.x0;
172         `CB_DRV.req_y     <= t.y0;
173         `CB_DRV.req_theta <= t.theta;
174         // Wait for the cycle in which valid *and* ready are both high. The
175         // clocking block guarantees that req_ready is the value sampled just
176         // before this edge, which is exactly the handshake semantics.
177         do `EDGE_DRV; while (`CB_DRV.req_ready != 1'b1);
178         `CB_DRV.req_valid <= 1'b0;
179     endtask
180
181     // Two independent threads: one feeds requests, one applies random
182     // back-pressure on the response channel. `fork ... join_none' inside a
183     // task is how a class spawns concurrency -- a VHDL process cannot be
184     // created at run time at all.
185     task automatic run();
186         fork
187             forever begin
188                 cordic_txn t;
189                 req_mbx.get(t);

```

```

190     send(t);
191 end
192 forever begin
193     if ($urandom_range(99) < stall_pct) begin
194         `CB_DRV.rsp_ready <= 1'b0;
195         for (int i = $urandom_range(3); i >= 0; i--) `EDGE_DRV;
196     end
197     `CB_DRV.rsp_ready <= 1'b1;
198     `EDGE_DRV;
199 end
200 join_none
201 endtask
202
203 endclass : cordic_drv
204
205
206 // =====
207 // 3. The monitor
208 //
209 // Strictly passive: it never drives a signal. It reconstructs
210 // transactions from the pins and publishes them. Keeping the monitor
211 // independent of the driver is what lets the same testbench be reused
212 // against a gate-level netlist, or against the real block inside a
213 // larger system.
214 // =====
215 class cordic_mon;
216
217     virtual cordic_if vif;
218     mailbox #(cordic_txn) obs_mbx;      // to the scoreboard
219     mailbox #(cordic_txn) cov_mbx;      // to the coverage collector
220
221     // Requests that have been accepted but whose result has not come back.
222     protected cordic_txn pending[$];
223
224     function new(virtual cordic_if vif,
225                 mailbox #(cordic_txn) obs_mbx,
226                 mailbox #(cordic_txn) cov_mbx);
227         this.vif = vif;
228         this.obs_mbx = obs_mbx;
229         this.cov_mbx = cov_mbx;
230     endfunction
231
232     task automatic run();
233         forever begin
234             `EDGE_MON;
235             if (vif.rst_n !== 1'b1) continue;
236
237             // Request accepted -> remember what went in.
238             if (`CB_MON.req_valid === 1'b1 && `CB_MON.req_ready === 1'b1) begin
239                 cordic_txn t = new();
240                 t.x0 = `CB_MON.req_x;
241                 t.y0 = `CB_MON.req_y;
242                 t.theta = `CB_MON.req_theta;
243                 pending.push_back(t);
244             end
245
246             // Response accepted -> complete the oldest outstanding request.
247             if (`CB_MON.rsp_valid === 1'b1 && `CB_MON.rsp_ready === 1'b1) begin
248                 cordic_txn t;
249                 if (pending.size() == 0) begin
250                     $error("monitor: response with no outstanding request");
251                 end else begin
252                     t = pending.pop_front();

```

```

253         t.x = `CB_MON.rsp_x;
254         t.y = `CB_MON.rsp_y;
255         obs_mbx.put(t);
256         cov_mbx.put(t);
257     end
258 end
259 end
260 endtask
261
262 endclass : cordic_mon
263
264
265 // =====
266 // 4. The scoreboard and its reference model
267 //
268 // The reference model is written at the level of the *specification*
269 // (real-valued trigonometry), not of the implementation. A model that
270 // re-implements the RTL will happily reproduce the RTL's bugs.
271 // =====
272 class cordic_sb;
273
274     mailbox #(cordic_txn) obs_mbx;
275     int unsigned n_checked = 0;
276     int unsigned n_failed = 0;
277     real        worst_err = 0.0;
278
279     function new(mailbox #(cordic_txn) obs_mbx);
280         this.obs_mbx = obs_mbx;
281     endfunction
282
283     // The golden model. Note that it says nothing about micro-rotations,
284     // guard bits or quadrant folding: it states what the block is supposed
285     // to compute.
286     function automatic void predict(input cordic_txn t,
287                                     output real x_ref, output real y_ref);
288         real th = q_to_real(t.theta, QA);
289         real xr = q_to_real(t.x0,   QF);
290         real yr = q_to_real(t.y0,   QF);
291         x_ref = K_GAIN * (xr * $cos(th) - yr * $sin(th));
292         y_ref = K_GAIN * (xr * $sin(th) + yr * $cos(th));
293     endfunction
294
295     task automatic run();
296         forever begin
297             cordic_txn t;
298             real x_ref, y_ref, ex, ey;
299             obs_mbx.get(t);
300             predict(t, x_ref, y_ref);
301
302             ex = (q_to_real(t.x, QF) - x_ref) * (2.0 ** QF); // error in LSB
303             ey = (q_to_real(t.y, QF) - y_ref) * (2.0 ** QF);
304             if (ex < 0.0) ex = -ex;
305             if (ey < 0.0) ey = -ey;
306             if (ex > worst_err) worst_err = ex;
307             if (ey > worst_err) worst_err = ey;
308
309             n_checked++;
310             if (ex > real'(TOL_LSB) || ey > real'(TOL_LSB)) begin
311                 n_failed++;
312                 $error({"MISMATCH %s\n          got  x=%0d y=%0d\n",
313                        "          want x=%.1f y=%.1f (err %.2f / %.2f LSB)",
314                        t.convert2string(), t.x, t.y,
315                        x_ref * (2.0 ** QF), y_ref * (2.0 ** QF), ex, ey});

```

```

316     end
317     end
318 endtask
319
320 function void report();
321     $display("-----");
322     $display(" scoreboard: %0d transactions checked, %0d failed",
323             n_checked, n_failed);
324     $display(" worst error: %.2f LSB (tolerance %0d LSB)",
325             worst_err, TOL_LSB);
326     $display("-----");
327     if (n_failed == 0 && n_checked > 0) $display(" *** TEST PASSED ***");
328     else $display(" *** TEST FAILED ***");
329 endfunction
330
331 endclass : cordic_sb
332
333
334 // =====
335 // 5. Functional coverage
336 //
337 // Coverage answers "what did I actually exercise?", which is a different
338 // question from "did it work?". VHDL has no equivalent construct: the
339 // usual workaround is counting in a process and printing a table.
340 // =====
341 class cordic_cov;
342
343     mailbox #(cordic_txn) cov_mbx;
344
345     `ifndef VERILATOR
346         angle_t sampled_theta;
347         data_t sampled_x0;
348
349         covergroup cg_stimulus;
350             option.per_instance = 1;
351
352             cp_quadrant : coverpoint sampled_theta {
353                 bins q3_neg = {[ -PI_EXT      : -PI_EXT/2 - 1]};
354                 bins q4_neg = {[ -PI_EXT/2    : -1]};
355                 bins zero   = {0};
356                 bins q1_pos = {[ 1           : PI_EXT/2 - 1]};
357                 bins q2_pos = {[ PI_EXT/2     : PI_EXT]};
358             }
359             cp_sign_x : coverpoint sampled_x0 {
360                 bins neg = {[ -32768 : -1]};
361                 bins zero = {0};
362                 bins pos = {[ 1 : 32767]};
363             }
364             x_quadrant_sign : cross cp_quadrant, cp_sign_x;
365         endgroup
366     `endif
367
368     function new(mailbox #(cordic_txn) cov_mbx);
369         this.cov_mbx = cov_mbx;
370     `ifndef VERILATOR
371         cg_stimulus = new();
372     `endif
373     endfunction
374
375     task automatic run();
376         forever begin
377             cordic_txn t;
378             cov_mbx.get(t);

```

```

379 `ifndef VERILATOR
380     sampled_theta = t.theta;
381     sampled_x0    = t.x0;
382     cg_stimulus.sample();
383 `endif
384     end
385 endtask
386
387 function void report();
388 `ifndef VERILATOR
389     $display(" functional coverage: %.1f %%", cg_stimulus.get_coverage());
390 `else
391     $display(" functional coverage: not collected (Verilator build)");
392 `endif
393 endfunction
394
395 endclass : cordic_cov
396
397 // =====
398 // 6. The environment
399 //
400 //
401 // Construction, connection and start-up, in that order.  UVM calls these
402 // build_phase, connect_phase and run_phase and calls them for you; here
403 // you call them yourself, which is one fewer thing to learn while the
404 // structure is still new.
405 // =====
406 class cordic_env;
407
408     virtual cordic_if vif;
409
410     mailbox #(cordic_txn) req_mbx;
411     mailbox #(cordic_txn) obs_mbx;
412     mailbox #(cordic_txn) cov_mbx;
413
414     cordic_drv drv;
415     cordic_mon mon;
416     cordic_sb sb;
417     cordic_cov cov;
418
419     function new(virtual cordic_if vif);
420         this.vif = vif;
421         // "build"
422         req_mbx = new();
423         obs_mbx = new();
424         cov_mbx = new();
425         drv     = new(vif, req_mbx);
426         mon     = new(vif, obs_mbx, cov_mbx);
427         sb      = new(obs_mbx);
428         cov     = new(cov_mbx);
429     endfunction
430
431     task automatic run();
432         fork
433             mon.run();
434             sb.run();
435             cov.run();
436         join_none
437             drv.run();
438         endtask
439
440     // Stimulus generation lives in the test, not in the environment: the
441     // environment is reusable, the stimulus is not.

```

```

442     task automatic send_random(int unsigned n);
443     repeat (n) begin
444         cordic_txn t = new();
445         t.do_randomize();
446         req_mbx.put(t);
447     end
448 endtask
449
450 task automatic send_directed(data_t x0, data_t y0, angle_t theta);
451     cordic_txn t = new();
452     t.x0 = x0; t.y0 = y0; t.theta = theta;
453     req_mbx.put(t);
454 endtask
455
456 // Wait until every transaction that was sent has been checked.
457 task automatic wait_drain(int unsigned expected, int unsigned timeout_cyc);
458     int unsigned waited = 0;
459     while (sb.n_checked < expected && waited < timeout_cyc) begin
460         `EDGE_MON;
461         waited++;
462     end
463     if (sb.n_checked < expected)
464         $fatal(1, "timeout: only %0d of %0d transactions completed",
465             sb.n_checked, expected);
466 endtask
467
468 function void report();
469     sb.report();
470     cov.report();
471 endfunction
472
473 endclass : cordic_env
474
475 endpackage : tb_cordic_pkg

```

Listing 1.8. `tb_cordic_pkg.sv` — the layered class testbench, complete.

```

1 // =====
2 // tb_cordic_top.sv -- the top level of the SystemVerilog testbench.
3 //
4 // The module has no ports. Its whole job is to exist so that there is
5 // somewhere to put the clock, the interface instance and the DUT, and then
6 // to hand a virtual interface to the class-based world in tb_cordic_pkg.
7 //
8 // Run with QuestaSim:
9 //     vlib work
10 //     vlog -sv cordic_pkg.sv cordic_if.sv cordic_rot.sv \
11 //         tb_cordic_pkg.sv tb_cordic_top.sv
12 //     vsim -c -sv_seed random -do "run -all; quit" tb_cordic_top
13 //
14 // Run with Verilator (no constraints, no coverage -- see Chapter 8):
15 //     $ verilator --binary --timing -j 0 \
16 //         cordic_pkg.sv cordic_if.sv cordic_rot.sv \
17 //         tb_cordic_pkg.sv tb_cordic_top.sv --top-module tb_cordic_top
18 //     ./obj_dir/Vtb_cordic_top
19 // =====
20 `timescale 1ns/1ps
21
22 module tb_cordic_top;
23
24     import cordic_pkg::*;
25     import tb_cordic_pkg::*;
26
27     // -----
28     // Clock. `always #5 clk = ~clk' is the whole of VHDL's
29     //     clk <= not clk after 5 ns;
30     // The variable must be initialised: an uninitialised `logic' is X, and
31     // ~X is X, so a testbench whose clock never starts is a classic first-day
32     // bug.
33     // -----
34     logic clk = 1'b0;
35     always #5 clk = ~clk;           // 100 MHz
36
37     // -----
38     // The pin bundle and the DUT.
39     // -----
40     cordic_if vif (.clk(clk));
41
42     cordic_rot #(.ITER(N)) u_dut (
43         .clk_i      (clk),
44         .rst_ni     (vif.rst_n),
45         .req_valid_i (vif.req_valid),
46         .req_ready_o (vif.req_ready),
47         .req_x_i     (vif.req_x),
48         .req_y_i     (vif.req_y),
49         .req_theta_i (vif.req_theta),
50         .rsp_valid_o (vif.rsp_valid),
51         .rsp_ready_i (vif.rsp_ready),
52         .rsp_x_o     (vif.rsp_x),
53         .rsp_y_o     (vif.rsp_y)
54     );
55
56     // -----
57     // The test.
58     //
59     // Note the shape of it: build, start, stimulate, drain, report, finish.
60     // Chapter 7 shows the same six steps written as UVM phases.
61     // -----
62     cordic_env env;
63

```

```

64  int unsigned n_random = 200;
65  int unsigned seed      = 1;
66
67  initial begin
68      // Plusargs are the SystemVerilog answer to a VHDL generic on a
69      // testbench: they let you change the run without recompiling.
70      void'($value$plusargs("NRANDOM=%d", n_random));
71      if ($value$plusargs("SEED=%d", seed)) begin
72          // $urandom with an argument seeds the thread's generator.
73          void'($urandom(seed));
74      end
75
76      $display("=== cordic_rot testbench =====");
77      $display(" ITER=%0d  DW=%0d.Q%0d  AW=%0d.Q%0d  random=%0d",
78              N, DW, QF, AW, QA, n_random);
79
80      env = new(vif);
81      env.run();
82      env.drv.reset();
83
84      // ---- directed corners first -----
85      // These are the values a human would think of. The random stream
86      // below is unlikely to hit them exactly, and they are exactly where a
87      // quadrant-folding bug lives.
88      env.send_directed(angle_t'(X0_UNIT), 16'sd0, 16'sd0); // 0
89      env.send_directed(angle_t'(X0_UNIT), 16'sd0, angle_t'(PI_EXT));
90      env.send_directed(angle_t'(X0_UNIT), 16'sd0, angle_t'(-PI_EXT));
91      env.send_directed(angle_t'(X0_UNIT), 16'sd0, angle_t'(PI_EXT/2));
92      env.send_directed(angle_t'(X0_UNIT), 16'sd0, angle_t'(-PI_EXT/2));
93      env.send_directed(angle_t'(X0_UNIT), 16'sd0, angle_t'(PI_EXT/4));
94      env.send_directed(16'sd0, angle_t'(X0_UNIT), angle_t'(PI_EXT/2));
95      env.send_directed(16'sd0, 16'sd0, angle_t'(PI_EXT/3));
96
97      // ---- then the random stream -----
98      env.send_random(n_random);
99
100     // ---- wait for everything to come back -----
101     // 8 directed + n_random transactions, each taking N+2 cycles plus the
102     // random back-pressure. The generous timeout is a watchdog, not an
103     // expectation.
104     env.wait_drain(8 + n_random, 200 * (8 + n_random) + 1000);
105
106     env.report();
107     $finish;
108 end
109
110 // -----
111 // A global watchdog. Without one, a deadlocked DUT gives you a simulation
112 // that runs until you notice, instead of a test that fails.
113 // -----
114 initial begin
115     #5ms;
116     $fatal(1, "global timeout: the testbench did not finish");
117 end
118
119 // -----
120 // Waveforms, on demand.
121 // -----
122 initial begin
123     if ($test$plusargs("WAVES")) begin
124         $dumpfile("tb_cordic.vcd");
125         $dumpvars(0, tb_cordic_top);
126     end

```

```
127     end
128
129 endmodule : tb_cordic_top
```

Listing 1.9. `tb_cordic_top.sv` — the top level: clock, interface, DUT, test.

1.10 The C++ Testbenches

Three, in increasing order of structure. All three drive the same Verilator-compiled model of the SystemVerilog RTL.

1.10.1 Direct: `tb_cordic.cpp`

Listing 1.10 is the shortest thing that does a real job: a clock loop, a handshake, a `<cmath>` reference and an error counter. Two hundred transactions, a tolerance check, an exit code.

The part to read carefully is the cycle:

```

1 void tick() {
2     dut->clk_i = 0;
3     dut->eval();           // settle: pre-edge values are now valid
4     ctx->timeInc(5000);    // 5 ns at 1 ps precision
5     dut->clk_i = 1;
6     dut->eval();           // the rising edge
7     ctx->timeInc(5000);
8     ++cycles_;
9 }
```

Set the inputs while the clock is low, call `eval()` so the combinational cone settles, and only then raise the clock. That sequence is the C++ spelling of “sample in the Preponed region”. Read `req_ready_o` after the low-phase `eval()` and you get the value the DUT will act on at the edge; read it after the high-phase `eval()` and you get next cycle’s, which is a bug you will spend an hour on.

△ Trap: `timeInc` counts in the context’s time precision

A `VerilatedContext` defaults to 1 ps precision, so a 5 ns half period is `timeInc(5000)`, not `timeInc(5)`. Get it wrong and the logic is unaffected — nothing in the design depends on absolute time — but every timestamp in the log and in the VCD is out by a factor of a thousand, and the waveform lines up with nothing.

This is not a hypothetical. In the sources shipped with this book, `tb_cordic.cpp` uses `timeInc(5)` while `tb_vectors.cpp` and `main_uvmlike.cpp` use `timeInc(5000)`. The second pair is right. The first is a real bug, found while writing this chapter and left in place so that you can see what it looks like: nothing fails, nothing warns, and the reported simulation time is wrong.

```

$ make cpp
-----
200 transactions, 0 failures, worst error 2.68 LSB
3206 clock cycles simulated
*** TEST PASSED ***
```

1.10.2 Bit-exact: `tb_vectors.cpp`

Listing 1.11 does something the other testbenches cannot: it compares against the Python model *exactly*. The vectors arrive as a generated header, so a stale table is a compile error rather than a mystery.

```

1 #include "cordic_vectors.h"           // generated by gen_vectors.py
2 ...
```

```
3 if (got_x != v.x || got_y != v.y) { ... ++failures; }
```

The other thing this file exists to demonstrate is sign extension:

```
1 static inline int16_t as_signed(uint16_t raw) {
2     return static_cast<int16_t>(raw);
3 }
```

△ Trap: Verilator ports carry no signedness

A 16-bit port becomes a `uint16_t` (`SData`). `logic signed [15:0]` and `logic [15:0]` produce the same C++ type. If you read `rsp_x_o` and compare it to a negative expected value without the cast, every positive result matches and every negative one fails — or, worse, you compare two unsigned values and everything passes. This is the single most common bug in a first C++ testbench.

```
$ make bitexact
-----
bit-exact check against the Python model
vectors generated with seed 1
315 checked, 0 mismatches, 5046 clock cycles
*** TEST PASSED ***
-----
```

A tolerance check and a bit-exact check fail on different things. The tolerance check fails when the mathematics is wrong: a missing iteration, a wrong sign, a bad table entry. The bit-exact check fails when a *rounding decision* has drifted: a shift that rounds toward zero instead of toward $-\infty$, a guard bit dropped one stage early, a saturation limit off by one. Neither subsumes the other, and a design of this kind wants both.

1.10.3 UVM-shaped: `svtlm.h`

The *Simulation Cookbook* builds the framework; here is the CORDIC sitting in it. The component tree is real, printed by the running program:

```
$ make uvmlike
[test.env.agt.seq      ] will generate 200 transactions
--- component tree ---
test (CordicTest)
  env (CordicEnv)
    agt (CordicAgent)
      seq (CordicSeq)
      drv (CordicDrv)
      mon (CordicMon)
      sb (CordicSb)
      cov (CordicCov)
--- run ---
--- report ---
[test.env.agt.drv      ] drove 200 transactions
[test.env.sb           ] 200 checked, 0 failed, worst 2.68 LSB (tol 12)
[test.env.cov          ] theta bins hit 4/5  [62 47 0 46 45]
*** TEST PASSED ***
```

Compare that tree with the UVM hierarchy in the *Verification Cookbook*. It is the same tree, with the same names, built by a factory from a string and configured from a key-value store, in about two hundred lines of header-only C++ and no library at all.

Look at the coverage line, which is the most interesting output in this chapter: [62 47 0 46 45]. Five bins over the angle range, and the middle one — θ exactly zero — was never hit in two hundred random transactions. Of course it was not: the probability of a uniform draw over 51 473 values landing on zero is 2×10^{-5} . That is what a coverage hole looks like, and the fix is the directed list at the top of Listing 1.9, which sends $\theta = 0$ deliberately.

Random stimulus will not find the corner you can name. Name it, send it, and use coverage to tell you about the ones you could not name.

```

1 // tb_cordic.cpp -- a self-checking C++ testbench for cordic_rot.
2 #include "Vcordic_rot.h"
3 #include "verilated.h"
4 #if VM_TRACE_FST
5 # include "verilated_fst_c.h"
6 using TraceC = VerilatedFstC;
7 static const char* TRACE_FILE = "cordic.fst";
8 #else
9 # include "verilated_vcd_c.h"
10 using TraceC = VerilatedVcdC;
11 static const char* TRACE_FILE = "cordic.vcd";
12 #endif
13
14 #include <cmath>
15 #include <stdint>
16 #include <stdio>
17 #include <random>
18 #include <cstring>
19 #include <cstdlib>
20
21 static const double K_GAIN = 1.6467602581210656;
22 static const int    QF = 14, QA = 13;
23 static const int    PI_EXT = 25736;    // round( $\pi * 2^{QA}$ )
24 static const double TOL_LSB = 12.0;
25
26 struct Txn {
27     int16_t x0, y0, theta;    // stimulus
28     int16_t x, y;            // observed
29 };
30
31 class CordicTb {
32 public:
33     VerilatedContext ctx;
34     Vcordic_rot*     dut;
35     TraceC*          tfp = nullptr;
36     uint64_t          cycles = 0;
37     unsigned          checked = 0, failed = 0;
38     double            worst = 0.0;
39
40     CordicTb(int argc, char** argv) {
41         ctx.commandArgs(argc, argv);
42         dut = new Vcordic_rot{&ctx};
43         dut->clk_i = 0; dut->rst_ni = 0;
44         dut->req_valid_i = 0; dut->rsp_ready_i = 0;
45         dut->req_x_i = 0; dut->req_y_i = 0; dut->req_theta_i = 0;
46     }
47     ~CordicTb() { closeTrace(); delete dut; }
48
49     void openTrace(const char* name) {
50         ctx.traceEverOn(true);
51         tfp = new TraceC;
52         dut->trace(tfp, 99);
53         tfp->open(name);
54     }
55     void closeTrace() { if (tfp) { tfp->close(); delete tfp; tfp = nullptr; } }
56
57     // One clock period. Inputs written before the call are seen by the
58     // rising edge; outputs read after the call are the post-edge values.
59     void tick() {
60         dut->clk_i = 0; dut->eval();
61         if (tfp) tfp->dump(ctx.time());
62         ctx.timeInc(5);
63     }

```

```

64     dut->clk_i = 1; dut->eval();
65     if (tfp) tfp->dump(ctx.time());
66     ctx.timeInc(5);
67     ++cycles;
68 }
69 // The combinational outputs of the current cycle, before the edge.
70 void settle() { dut->clk_i = 0; dut->eval(); }
71
72 void reset(int n = 5) {
73     dut->rst_ni = 0;
74     for (int i = 0; i < n; ++i) tick();
75     dut->rst_ni = 1;
76     tick();
77 }
78
79 void check(const Txn& t) {
80     double th = double(t.theta) / (1 << QA);
81     double xr = double(t.x0) / (1 << QF);
82     double yr = double(t.y0) / (1 << QF);
83     double rx = K_GAIN * (xr * std::cos(th) - yr * std::sin(th));
84     double ry = K_GAIN * (xr * std::sin(th) + yr * std::cos(th));
85     double ex = std::fabs(double(t.x) - rx * (1 << QF));
86     double ey = std::fabs(double(t.y) - ry * (1 << QF));
87     if (ex > worst) worst = ex;
88     if (ey > worst) worst = ey;
89     ++checked;
90     if (ex > TOL_LSB || ey > TOL_LSB) {
91         ++failed;
92         printf("MISMATCH x0=%d y0=%d th=%d : got (%d,%d) "
93             "want (%.1f,%.1f) err %.2f/%.2f LSB\n",
94             t.x0, t.y0, t.theta, t.x, t.y,
95             rx * (1 << QF), ry * (1 << QF), ex, ey);
96     }
97 }
98 };
99
100 int main(int argc, char** argv) {
101     CordicTb tb(argc, argv);
102
103     int n = 200;
104     bool trace = false, bp = false;
105     for (int i = 1; i < argc; ++i) {
106         if (!strcmp(argv[i], "+n="), 3)) n = atoi(argv[i] + 3);
107         if (!strcmp(argv[i], "+trace")) trace = true;
108         if (!strcmp(argv[i], "+bp")) bp = true;
109     }
110     if (trace) tb.openTrace	TRACE_FILE);
111
112     tb.reset();
113     tb.dut->rsp_ready_i = 1;
114
115     std::mt19937 rng(1);
116     std::uniform_int_distribution<int> dth(-PI_EXT, PI_EXT);
117     std::uniform_int_distribution<int> dxy(-6000, 6000);
118
119     for (int i = 0; i < n; ++i) {
120         Txn t;
121         t.theta = int16_t(dth(rng));
122         t.x0 = int16_t(dxy(rng));
123         t.y0 = int16_t(dxy(rng));
124
125         // --- drive the request and hold it until it is accepted -----
126         tb.dut->req_valid_i = 1;

```

```

127     tb.dut->req_x_i    = uint16_t(t.x0);
128     tb.dut->req_y_i    = uint16_t(t.y0);
129     tb.dut->req_theta_i = uint16_t(t.theta);
130     bool accepted;
131     do {
132         tb.settle();
133         accepted = tb.dut->req_ready_o;
134         tb.tick();
135     } while (!accepted);
136     tb.dut->req_valid_i = 0;
137
138     // --- wait for the response, with optional back-pressure -----
139     int guard = 0;
140     bool done = false;
141     while (!done && guard++ < 1000) {
142         if (bp) tb.dut->rsp_ready_i = (rng() % 4) != 0;
143         tb.settle();
144         if (tb.dut->rsp_valid_o && tb.dut->rsp_ready_i) {
145             t.x = int16_t(tb.dut->rsp_x_o);
146             t.y = int16_t(tb.dut->rsp_y_o);
147             done = true;
148         }
149         tb.tick();
150     }
151     if (!done) { printf("TIMEOUT\n"); return 2; }
152     tb.check(t);
153 }
154
155 tb.dut->final();
156 tb.closeTrace();
157 #if VM_COVERAGE
158 Verilated::mkdir("logs");
159 tb.ctx.coveragep()->write("logs/coverage.dat");
160 #endif
161 printf("-----\n");
162 printf(" %u transactions, %u failures, worst error %.2f LSB\n",
163        tb.checked, tb.failed, tb.worst);
164 printf(" %llu clock cycles simulated\n", (unsigned long long)tb.cycles);
165 printf(" *** TEST %s ***\n", tb.failed ? "FAILED" : "PASSED");
166 return tb.failed ? 1 : 0;
167 }

```

Listing 1.10. `tb_cordic.cpp` — the direct C++ testbench.

```

1 // =====
2 // tb_vectors.cpp -- a bit-exact C++ testbench driven by generated vectors.
3 //
4 // The other testbenches compare against a floating point model with a
5 // tolerance, which checks that the block computes a rotation. This one
6 // compares against the Python model bit for bit, which checks that the RTL
7 // and the model implement the *same* fixed-point algorithm. Both checks are
8 // worth having, and they fail in different ways:
9 //
10 // * a tolerance check fails when the maths is wrong;
11 // * a bit-exact check fails when a rounding mode, a guard bit or a
12 //   truncation has drifted between the model and the RTL.
13 //
14 // The vectors arrive as a generated header, so a stale table is a compile
15 // error rather than a mysterious mismatch:
16 //
17 // make vectors ES make bitexact
18 // =====
19 #include "Vcordic_rot.h"
20 #include "cordic_vectors.h"
21 #include "verilated.h"
22
23 #include <cstdio>
24 #include <cstdlib>
25
26 // -----
27 // A thin wrapper around the Verilated model.
28 //
29 // Note the shape of the clock: set the inputs while the clock is low, call
30 // eval() so that the combinational cone settles, and only then raise the
31 // clock. This is the C++ spelling of "sample in the Preponed region".
32 // -----
33 class Bench {
34 public:
35     explicit Bench(VerilatedContext* ctx) : ctx_(ctx), dut_(new Vcordic_rot) {}
36     ~Bench() { dut_>final(); delete dut_; }
37
38     Vcordic_rot* dut() { return dut_; }
39     uint64_t cycles() const { return cycles_; }
40
41     void reset(int n = 5) {
42         dut_>clk_i = 0;
43         dut_>rst_ni = 0;
44         dut_>req_valid_i = 0;
45         dut_>rsp_ready_i = 1;
46         dut_>req_x_i = dut_>req_y_i = dut_>req_theta_i = 0;
47         for (int i = 0; i < n; ++i) tick();
48         dut_>rst_ni = 1;
49         tick();
50     }
51
52     // One clock cycle. Inputs must already be set when this is called.
53     void tick() {
54         dut_>clk_i = 0;
55         dut_>eval();
56         ctx_>timeInc(5000); // 5 ns at 1 ps precision
57         dut_>clk_i = 1;
58         dut_>eval();
59         ctx_>timeInc(5000);
60         ++cycles_;
61     }
62
63 private:

```

```

64 VerilatedContext* ctx_;
65 Vcordic_rot* dut_;
66 uint64_t cycles_ = 0;
67 };
68
69 // -----
70 // Verilator gives a 16-bit port the C type uint16_t (SData). It knows
71 // nothing about signedness, so the sign extension is yours to do. Getting
72 // this wrong is the single most common bug in a C++ testbench, and it hides
73 // well: every positive result is correct.
74 // -----
75 static inline int16_t as_signed(uint16_t raw) {
76     return static_cast<int16_t>(raw);
77 }
78
79 int main(int argc, char** argv) {
80     VerilatedContext ctx;
81     ctx.commandArgs(argc, argv);
82
83     Bench b(&ctx);
84     Vcordic_rot* d = b.dut();
85     b.reset();
86
87     unsigned failures = 0;
88     unsigned checked = 0;
89
90     for (unsigned i = 0; i < kCordicVecCount; ++i) {
91         const CordicVec& v = kCordicVectors[i];
92
93         // ---- drive the request, holding it until req_ready_o is seen ----
94         d->req_x_i = static_cast<uint16_t>(v.x0);
95         d->req_y_i = static_cast<uint16_t>(v.y0);
96         d->req_theta_i = static_cast<uint16_t>(v.theta);
97         d->req_valid_i = 1;
98         unsigned guard = 0;
99         do {
100             d->clk_i = 0;
101             d->eval(); // settle, then look at req_ready_o
102             if (d->req_ready_o) break;
103             b.tick();
104         } while (++guard < 1000);
105         b.tick(); // the cycle in which it is accepted
106         d->req_valid_i = 0;
107
108         // ---- wait for the response ----
109         guard = 0;
110         while (guard++ < 1000) {
111             d->clk_i = 0;
112             d->eval();
113             if (d->rsp_valid_o) break;
114             b.tick();
115         }
116         if (guard >= 1000) {
117             std::fprintf(stderr, "vector %u: no response\n", i);
118             ++failures;
119             break;
120         }
121
122         const int16_t got_x = as_signed(d->rsp_x_o);
123         const int16_t got_y = as_signed(d->rsp_y_o);
124         b.tick(); // consume it (rsp_ready_i is tied high)
125
126         ++checked;

```

```

127     if (got_x != v.x || got_y != v.y) {
128         if (failures < 10) {
129             std::printf("MISMATCH vector %u:  x0=%d y0=%d theta=%d\n",
130                         "          got  x=%d y=%d\n",
131                         "          want x=%d y=%d\n",
132                         i, v.x0, v.y0, v.theta, got_x, got_y, v.x, v.y);
133         }
134         ++failures;
135     }
136 }
137
138 std::printf("-----\n");
139 std::printf(" bit-exact check against the Python model\n");
140 std::printf(" vectors generated with seed %u\n", kCordicVecSeed);
141 std::printf(" %u checked, %u mismatches, %llu clock cycles\n",
142             checked, failures,
143             static_cast<unsigned long long>(b.cycles()));
144 std::printf(" %s\n", failures ? "*** TEST FAILED ***" : "*** TEST PASSED ***");
145 std::printf("-----\n");
146 return failures ? 1 : 0;
147 }

```

Listing 1.11. `tb_vectors.cpp` — the bit-exact C++ testbench.

```

1 // =====
2 // svtlm.h -- a very small UVM-shaped verification framework in C++.
3 //
4 // Four ideas, and nothing else:
5 //   * Component    -- a named node in a tree, with build/connect/tick/report
6 //   * AnalysisPort-- one writer, many readers, TLM style
7 //   * Factory       -- create a component by type *name*, with overrides
8 //   * ConfigDb      -- a string-keyed store read during build()
9 // =====
10 #ifndef SVTLM_H_
11 #define SVTLM_H_
12
13 #include <cstdarg>
14 #include <cstdio>
15 #include <cstdlib>
16 #include <functional>
17 #include <map>
18 #include <memory>
19 #include <string>
20 #include <vector>
21
22 namespace svtlm {
23
24 // -----
25 // Configuration store. The UVM equivalent is uvm_config_db#(T).
26 // A key is "<component path>.<field>"; "*" matches any path.
27 // -----
28 class ConfigDb {
29 public:
30     static ConfigDb& get() { static ConfigDb db; return db; }
31
32     void setInt(const std::string& path, const std::string& field, int v) {
33         ints_[path + "." + field] = v;
34     }
35     void setStr(const std::string& path, const std::string& field,
36                const std::string& v) {
37         strs_[path + "." + field] = v;
38     }
39     bool getInt(const std::string& path, const std::string& field, int& v) const {
40         auto it = ints_.find(path + "." + field);
41         if (it == ints_.end()) it = ints_.find("*." + field);
42         if (it == ints_.end()) return false;
43         v = it->second;
44         return true;
45     }
46     int getIntOr(const std::string& path, const std::string& field, int dflt) const {
47         int v; return getInt(path, field, v) ? v : dflt;
48     }
49     bool getStr(const std::string& path, const std::string& field,
50                std::string& v) const {
51         auto it = strs_.find(path + "." + field);
52         if (it == strs_.end()) it = strs_.find("*." + field);
53         if (it == strs_.end()) return false;
54         v = it->second;
55         return true;
56     }
57 private:
58     std::map<std::string, int>      ints_;
59     std::map<std::string, std::string> strs_;
60 };
61
62 // -----
63 // Analysis port. uvm_analysis_port#(T) with the boilerplate removed:

```

```

64 // a subscriber is any callable, so a lambda that forwards to a method is
65 // enough and no *_imp class is needed.
66 // -----
67 template <typename T>
68 class AnalysisPort {
69 public:
70     using Sub = std::function<void(const T&)>;
71     void connect(Sub s) { subs_.push_back(std::move(s)); }
72     void write(const T& t) const { for (const auto& s : subs_) s(t); }
73     size_t nsubs() const { return subs_.size(); }
74 private:
75     std::vector<Sub> subs_;
76 };
77
78 // -----
79 // Component: the phase machinery.
80 //
81 // UVM's build_phase is top-down, connect_phase bottom-up, run_phase
82 // concurrent. Here build() is top-down (a parent creates its children),
83 // connect() is bottom-up, tick() is called once per clock on every
84 // component in construction order, and report() runs at the end.
85 // -----
86 class Component {
87 public:
88     Component(std::string name, Component* parent)
89         : name_(std::move(name)), parent_(parent) {
90         if (parent_) parent_->children_.push_back(this);
91     }
92     virtual ~Component() = default;
93
94     const std::string& name() const { return name_; }
95     Component* parent() const { return parent_; }
96
97     std::string fullName() const {
98         return parent_ ? parent_->fullName() + "." + name_ : name_;
99     }
100
101     // ---- the phases, overridden by the user -----
102     virtual void build() {}
103     virtual void connect() {}
104     virtual void tick() {}
105     virtual void report() {}
106
107     // ---- the phase walkers, called by the test -----
108     void buildAll() {
109         build();
110         // build() may have created children, so index the vector by hand.
111         for (size_t i = 0; i < children_.size(); ++i) children_[i]->buildAll();
112     }
113     void connectAll() { // bottom-up, like UVM
114         for (auto* c : children_) c->connectAll();
115         connect();
116     }
117     void tickAll() {
118         tick();
119         for (auto* c : children_) c->tickAll();
120     }
121     void reportAll() {
122         for (auto* c : children_) c->reportAll();
123         report();
124     }
125     void printTree(int depth = 0) const {
126         printf("%*s%s (%s)\n", depth * 2, "", name_.c_str(), typeName().c_str());

```

```

127     for (const auto* c : children_) c->printTree(depth + 1);
128 }
129 virtual std::string typeName() const { return "Component"; }
130
131 // ---- reporting -----
132 void info(const char* fmt, ...) const;
133
134 int cfgInt(const std::string& field, int dflt) const {
135     return ConfigDb::get().getIntOr(fullName(), field, dflt);
136 }
137
138 protected:
139     std::vector<Component*> children_;
140 private:
141     std::string name_;
142     Component* parent_;
143 };
144
145 inline void Component::info(const char* fmt, ...) const {
146     va_list ap;
147     va_start(ap, fmt);
148     printf("[%s] ", fullName().c_str());
149     vprintf(fmt, ap);
150     printf("\n");
151     va_end(ap);
152 }
153
154 // -----
155 // Factory.  uvm_factory with one type of override: by type name.
156 // -----
157 class Factory {
158 public:
159     using Ctor = std::function<Component*(const std::string&, Component*)>;
160
161     static Factory& get() { static Factory f; return f; }
162
163     void reg(const std::string& type, Ctor c) { ctors_[type] = std::move(c); }
164     void setTypeOverride(const std::string& orig, const std::string& repl) {
165         overrides_[orig] = repl;
166     }
167     Component* create(const std::string& type, const std::string& name,
168                     Component* parent) {
169         std::string t = type;
170         auto ov = overrides_.find(t);
171         if (ov != overrides_.end()) t = ov->second;
172         auto it = ctors_.find(t);
173         if (it == ctors_.end()) {
174             printf("FATAL: factory has no type '%s'\n", t.c_str());
175             abort();
176         }
177         return it->second(name, parent);
178     }
179     template <typename T>
180     T* createAs(const std::string& type, const std::string& name,
181               Component* parent) {
182         return static_cast<T*>(create(type, name, parent));
183     }
184 private:
185     std::map<std::string, Ctor> ctors_;
186     std::map<std::string, std::string> overrides_;
187 };
188
189 // Registration helper: one static object per component type.

```

```

190 // The UVM counterpart is the `uvm_component_utils macro.
191 template <typename T>
192 struct Registrar {
193     explicit Registrar(const std::string& type) {
194         Factory::get().reg(type, [](const std::string& n, Component* p) {
195             return static_cast<Component*>(new T(n, p));
196         });
197     }
198 };
199
200 #define SVTLM_REGISTER(TYPE) \
201     std::string typeName() const override { return #TYPE; } \
202     static inline svtlm::Registrar<TYPE> svtlm_reg_{#TYPE}
203
204 } // namespace svtlm
205 #endif // SVTLM_H_

```

Listing 1.12. `svtlm.h` — the UVM-shaped framework: Component, AnalysisPort, Factory, ConfigDb.

```

1 // =====
2 // cordic_env.h -- a UVM-shaped environment for cordic_rot, built on svtlm.h.
3 // =====
4 #ifndef CORDIC_ENV_H_
5 #define CORDIC_ENV_H_
6
7 #include "Vcordic_rot.h"
8 #include "svtlm.h"
9
10 #include <cmath>
11 #include <cstdint>
12 #include <deque>
13 #include <random>
14
15 using svtlm::AnalysisPort;
16 using svtlm::Component;
17 using svtlm::ConfigDb;
18 using svtlm::Factory;
19
20 static const double K_GAIN = 1.6467602581210656;
21 static const int    QF = 14, QA = 13, PI_EXT = 25736;
22
23 // -----
24 // 1. The transaction. A plain value type: no base class, no factory.
25 // -----
26 struct CordicTxn {
27     int16_t x0 = 0, y0 = 0, theta = 0;    // stimulus
28     int16_t x = 0, y = 0;                // observed response
29     unsigned id = 0;
30 };
31
32 // -----
33 // 2. The pin-level view. The C++ answer to a virtual interface: one struct
34 //    that every component holds a pointer to.
35 // -----
36 struct CordicVif {
37     Vcordic_rot* dut = nullptr;
38     uint64_t     cycle = 0;
39 };
40
41 // -----
42 // 3. Sequence: produces transactions into the driver's queue.
43 // -----
44 class CordicSeq : public Component {
45 public:
46     SVTLM_REGISTER(CordicSeq);
47     CordicSeq(const std::string& n, Component* p) : Component(n, p) {}
48
49     std::deque<CordicTxn>* q = nullptr;    // wired by the agent
50     unsigned n_total = 0, n_sent = 0;
51
52     void build() override {
53         n_total = cfgInt("n_txn", 200);
54         rng_.seed(cfgInt("seed", 1));
55         info("will generate %u transactions", n_total);
56     }
57     void tick() override {
58         if (!q || n_sent >= n_total || q->size() >= 2) return;
59         CordicTxn t;
60         t.id    = n_sent++;
61         t.theta = int16_t(dth_(rng_));
62         t.x0    = int16_t(dxy_(rng_));
63         t.y0    = int16_t(dxy_(rng_));

```

```

64     q->push_back(t);
65 }
66 bool done() const { return n_sent >= n_total; }
67
68 private:
69     std::mt19937 rng_{1};
70     std::uniform_int_distribution<int> dth_{-PI_EXT, PI_EXT};
71     std::uniform_int_distribution<int> dxy_{-6000, 6000};
72 };
73
74 // -----
75 // 4. Driver: an explicit two-state machine, one step per clock.
76 // -----
77 class CordicDrv : public Component {
78 public:
79     SVTLM_REGISTER(CordicDrv);
80     CordicDrv(const std::string& n, Component* p) : Component(n, p) {}
81
82     CordicVif*      vif = nullptr;
83     std::deque<CordicTxn> q;
84     unsigned        stall_pct = 0, n_driven = 0;
85
86     void build() override {
87         stall_pct = cfgInt("stall_pct", 0);
88         rng_.seed(cfgInt("seed", 2));
89     }
90     void tick() override {
91         Vcortic_rot* d = vif->dut;
92
93         // --- request channel -----
94         if (st_ == IDLE) {
95             if (!q.empty()) {
96                 const CordicTxn& t = q.front();
97                 d->req_valid_i = 1;
98                 d->req_x_i     = uint16_t(t.x0);
99                 d->req_y_i     = uint16_t(t.y0);
100                 d->req_theta_i = uint16_t(t.theta);
101                 st_ = DRIVING;
102             } else {
103                 d->req_valid_i = 0;
104             }
105         } else if (d->req_ready_o) { // handshake completes on this edge
106             q.pop_front();
107             ++n_driven;
108             d->req_valid_i = 0;
109             st_ = IDLE;
110         }
111
112         // --- response channel: random back-pressure -----
113         d->rsp_ready_i = (stall_pct == 0) || (int(rng_() % 100) >= int(stall_pct));
114     }
115     void report() override { info("drove %u transactions", n_driven); }
116
117 private:
118     enum State { IDLE, DRIVING } st_ = IDLE;
119     std::mt19937 rng_{2};
120 };
121
122 // -----
123 // 5. Monitor: passive, publishes on an analysis port.
124 // -----
125 class CordicMon : public Component {
126 public:

```

```

127     SVTLM_REGISTER(CordicMon);
128     CordicMon(const std::string& n, Component* p) : Component(n, p) {}
129
130     CordicVif*          vif = nullptr;
131     AnalysisPort<CordicTxn> ap;
132     unsigned            n_seen = 0;
133
134     void tick() override {
135         Vcortic_rot* d = vif->dut;
136         if (!d->rst_ni) { pending_.clear(); return; }
137
138         if (d->req_valid_i && d->req_ready_o) {
139             CordicTxn t;
140             t.x0 = int16_t(d->req_x_i);
141             t.y0 = int16_t(d->req_y_i);
142             t.theta = int16_t(d->req_theta_i);
143             t.id = n_seen++;
144             pending_.push_back(t);
145         }
146         if (d->rsp_valid_o && d->rsp_ready_i) {
147             if (pending_.empty()) { info("ERROR response with no request"); return; }
148             CordicTxn t = pending_.front();
149             pending_.pop_front();
150             t.x = int16_t(d->rsp_x_o);
151             t.y = int16_t(d->rsp_y_o);
152             ap.write(t);                      // TLM broadcast
153         }
154     }
155 private:
156     std::deque<CordicTxn> pending_;
157 };
158
159 // -----
160 // 6. Scoreboard: subscribes to the analysis port, owns the golden model.
161 // -----
162 class CordicSb : public Component {
163 public:
164     SVTLM_REGISTER(CordicSb);
165     CordicSb(const std::string& n, Component* p) : Component(n, p) {}
166
167     unsigned checked = 0, failed = 0;
168     double   worst = 0.0, tol = 12.0;
169
170     void build() override { tol = cfgInt("tol_lsb", 12); }
171
172     virtual void write(const CordicTxn& t) {
173         double th = double(t.theta) / (1 << QA);
174         double xr = double(t.x0) / (1 << QF), yr = double(t.y0) / (1 << QF);
175         double rx = K_GAIN * (xr * std::cos(th) - yr * std::sin(th)) * (1 << QF);
176         double ry = K_GAIN * (xr * std::sin(th) + yr * std::cos(th)) * (1 << QF);
177         double ex = std::fabs(double(t.x) - rx), ey = std::fabs(double(t.y) - ry);
178         worst = std::max(worst, std::max(ex, ey));
179         ++checked;
180         if (ex > tol || ey > tol) {
181             ++failed;
182             info("MISMATCH id=%u theta=%d got(%d,%d) want(%.1f,%.1f)",
183                 t.id, t.theta, t.x, t.y, rx, ry);
184         }
185     }
186     void report() override {
187         info("%u checked, %u failed, worst %.2f LSB (tol %.0f)",
188             checked, failed, worst, tol);
189     }

```

```

190 };
191
192 // A second implementation of the same role, selected by a factory override.
193 class CordicSbStrict : public CordicSb {
194 public:
195     SVTLM_REGISTER(CordicSbStrict);
196     CordicSbStrict(const std::string& n, Component* p) : CordicSb(n, p) {}
197     void build() override { CordicSb::build(); tol = 2.0; }
198     void report() override {
199         info("STRICT mode: %u checked, %u failed, worst %.2f LSB (tol %.0f)",
200             checked, failed, worst, tol);
201     }
202 };
203
204 // -----
205 // 7. Coverage collector: also a subscriber of the same analysis port.
206 // -----
207 class CordicCov : public Component {
208 public:
209     SVTLM_REGISTER(CordicCov);
210     CordicCov(const std::string& n, Component* p) : Component(n, p) {}
211
212     unsigned bins[5] = {0, 0, 0, 0, 0}; // quadrant of theta
213
214     void write(const CordicTxn& t) {
215         int th = t.theta;
216         if (th < -PI_EXT / 2) bins[0]++;
217         else if (th < 0) bins[1]++;
218         else if (th == 0) bins[2]++;
219         else if (th < PI_EXT / 2) bins[3]++;
220         else bins[4]++;
221     }
222     void report() override {
223         unsigned hit = 0;
224         for (unsigned b : bins) hit += (b != 0);
225         info("theta bins hit %u/5 [%u %u %u %u %u]",
226             hit, bins[0], bins[1], bins[2], bins[3], bins[4]);
227     }
228 };
229
230 // -----
231 // 8. Agent and environment: pure structure, created through the factory.
232 // -----
233 class CordicAgent : public Component {
234 public:
235     SVTLM_REGISTER(CordicAgent);
236     CordicAgent(const std::string& n, Component* p) : Component(n, p) {}
237
238     CordicVif* vif = nullptr;
239     CordicSeq* seq = nullptr;
240     CordicDrv* drv = nullptr;
241     CordicMon* mon = nullptr;
242
243     void build() override {
244         Factory& f = Factory::get();
245         seq = f.createAs<CordicSeq>("CordicSeq", "seq", this);
246         drv = f.createAs<CordicDrv>("CordicDrv", "drv", this);
247         mon = f.createAs<CordicMon>("CordicMon", "mon", this);
248         drv->vif = vif;
249         mon->vif = vif;
250     }
251     void connect() override { seq->q = &drv->q; }
252 };

```

```

253
254 class CordicEnv : public Component {
255     public:
256         SVTLM_REGISTER(CordicEnv);
257         CordicEnv(const std::string& n, Component* p) : Component(n, p) {}
258
259         CordicVif*   vif = nullptr;
260         CordicAgent* agt = nullptr;
261         CordicSb*    sb  = nullptr;
262         CordicCov*   cov = nullptr;
263
264         void build() override {
265             Factory& f = Factory::get();
266             agt = f.createAs<CordicAgent>("CordicAgent", "agt", this);
267             agt->vif = vif;
268             sb  = f.createAs<CordicSb>("CordicSb", "sb", this);
269             cov = f.createAs<CordicCov>("CordicCov", "cov", this);
270         }
271         void connect() override {
272             // TLM: one producer, two consumers, no knowledge of each other.
273             CordicSb* s = sb;
274             CordicCov* c = cov;
275             agt->mon->ap.connect([s](const CordicTxn& t) { s->write(t); });
276             agt->mon->ap.connect([c](const CordicTxn& t) { c->write(t); });
277         }
278     };
279
280 #endif // CORDIC_ENV_H_

```

Listing 1.13. `cordic_env.h` — the CORDIC components built on `svtlm.h`.

```

1 // =====
2 // main_uvmlike.cpp -- the "test" layer and the clock loop.
3 // =====
4 #include "cordic_env.h"
5 #include "verilated.h"
6
7 #include <cstring>
8
9 // -----
10 // The test: configures, builds, runs, reports. This is the only place that
11 // knows both the environment and the clock.
12 // -----
13 class CordicTest : public Component {
14 public:
15     SVTLM_REGISTER(CordicTest);
16     CordicTest(const std::string& n, Component* p) : Component(n, p) {}
17
18     CordicVif vif;
19     CordicEnv* env = nullptr;
20
21     void build() override {
22         env = Factory::get().createAs<CordicEnv>("CordicEnv", "env", this);
23         env->vif = &vif;
24     }
25
26     // The clock loop. Everything above is called from here, once per cycle.
27     void run(VerilatedContext& ctx, unsigned max_cycles) {
28         Vcordic_rot* d = vif.dut;
29         d->clk_i = 0; d->rst_ni = 0;
30         d->req_valid_i = 0; d->rsp_ready_i = 0;
31         d->req_x_i = d->req_y_i = d->req_theta_i = 0;
32
33         for (unsigned c = 0; c < max_cycles; ++c) {
34             d->clk_i = 0;
35             d->eval(); // settle: pre-edge values are now valid
36             if (c == 5) d->rst_ni = 1;
37             if (c > 5) tickAll(); // every component sees this cycle
38             ctx.timeInc(5000); // 5 ns, in 1 ps precision units
39             d->clk_i = 1;
40             d->eval(); // the rising edge
41             ctx.timeInc(5000);
42             vif.cycle = c;
43             if (finished()) break;
44         }
45     }
46     bool finished() const {
47         return env->agt->seq->done() && env->agt->drv->q.empty() &&
48             env->sb->checked >= env->agt->seq->n_total;
49     }
50     int status() const { return env->sb->failed ? 1 : 0; }
51 };
52
53 int main(int argc, char** argv) {
54     VerilatedContext ctx;
55     ctx.commandArgs(argc, argv);
56
57     // ---- configuration, before build() -----
58     ConfigDb& cfg = ConfigDb::get();
59     int n = 200;
60     for (int i = 1; i < argc; ++i) {
61         if (!strncmp(argv[i], "+n=", 3)) n = atoi(argv[i] + 3);
62         if (!strncmp(argv[i], "+stall=", 7))
63             cfg.setInt("stall_pct", atoi(argv[i] + 7));
64     }
65 }

```

```

64     if (!strcmp(argv[i], "+strict"))
65         Factory::get().setTypeOverride("CordicSb", "CordicSbStrict");
66     }
67     cfg.setInt("n", "n_txn", n);
68     cfg.setInt("test.env.sb", "tol_lsb", 12);
69
70     // ---- elaborate -----
71     Vcordic_rot dut{&ctx};
72     CordicTest test("test", nullptr);
73     test.vif.dut = &dut;
74
75     test.buildAll();
76     test.connectAll();
77     printf("--- component tree ---\n");
78     test.printTree();
79     printf("--- run ---\n");
80
81     test.run(ctx, 200u * unsigned(n) + 1000u);
82
83     dut.final();
84     printf("--- report ---\n");
85     test.reportAll();
86     printf("*** TEST %s ***\n", test.status() ? "FAILED" : "PASSED");
87     return test.status();
88 }

```

Listing 1.14. `main_uvmlike.cpp` — the test layer and the clock loop.

1.11 The Python Testbench

1.11.1 Generated everything

Listing 1.15 is one script with four outputs, and it is the practical face of the *Simulation Cookbook*:

- `vectors/cordic_vectors.txt` — a plain text vector file that any testbench can read;
- `verilator/cordic_vectors.h` — a C++ header with `constexpr` arrays, consumed by `tb_vectors.cpp`;
- `sv/cordic_vectors.svh` — the same table as SystemVerilog `localparam` arrays;
- `sv/cordic_atan_rom.svh` — the arctangent ROM as generated source, for the tools that cannot compute it themselves.

The vector file carries its own provenance, which matters more than it looks:

```
# cordic_rot test vectors
# generated 2026-09-05 06:19:35
# generator gen_vectors.py seed=1
# format DW=16 QF=14 AW=16 QA=13 N=14 GL=2 GM=2
# count 315
# x0 y0 theta xexp yexp
26dd 0000 0000 3fff 0001
26dd 0000 6488 c000 0001
```

Without the header, a vector file is an unreproducible blob that happens to be readable. With it, anyone can regenerate it and see whether it changed.

△ Trap: Comment characters depend on the reader

This file uses `#` because it is read by `$fgets/$sscanf`, by `std.textio` and by a C++ `ifstream`, all of which are happy to skip a line you tell them to skip. A file destined for `$readmemh` is different: `$readmemh` accepts only `//` and `/* */` comments, and a `#` in it is a syntax error — or, in some tools, silently the value zero. Decide who reads the file before you choose the comment character.

1.11.2 cocotb: the same test against both languages

Listing 1.16 is a testbench with no HDL in it at all. The top level of the simulation is the DUT; `cordic_model.py` is imported directly and used as a bit-exact reference; the driver, the monitor and the scoreboard are coroutines.

```
1 async def monitor(dut, queue, results, expected):
2     while len(results) < expected:
3         await RisingEdge(dut.clk_i)
4         if dut.rsp_valid_o.value == 1 and dut.rsp_ready_i.value == 1:
5             txn = queue.popleft()
6             txn.x = dut.rsp_x_o.value.to_signed()
7             txn.y = dut.rsp_y_o.value.to_signed()
8             results.append(txn)
```

△ Trap: .value is unsigned

`int(dut.rsp_x_o.value)` on the result `-16384` gives `49152`. The test `test_value_semantics` in Listing 1.16 exists purely to print all four spellings side by side so that you never have to rediscover this. In `cocotb 2.x` the signed conversion is `.to_signed()`; in `1.x` it was `.signed_integer`. The equivalent `1.x` spellings for the rest are `units=` rather than `unit=` on `Clock`, and `.binstr` rather than `str(v)`.

The headline is the Makefile. Two variables choose the language and the backend; nothing else in the flow moves:

```
ifeq ($(SV),1)
    SIM          := verilator
    TOPLEVEL_LANG := verilog
    VERILOG_SOURCES := $(ROOT)/sv/cordic_pkg.sv $(ROOT)/sv/cordic_rot.sv
else
    SIM          := questa
    TOPLEVEL_LANG := vhdl
    VHDL_SOURCES := $(ROOT)/vhdl/cordic_pkg.vhd $(ROOT)/vhdl/cordic_rot.vhd
    COMPILE_ARGS += -2008
endif
```

One Python file, two languages, two backends, no change to the test. `SIM = questa` reaches the VHDL through the FLI, the C interface `QuestaSim` exposes for exactly this purpose; `SIM = verilator` reaches the SystemVerilog through VPI. `COMPILE_ARGS` is passed to the analysis step, which is where a language-revision switch belongs. Put it in `SIM_ARGS` instead and it goes to the run, which does not understand it.

`cocotb` prints its own summary table, and it prints it identically whichever backend ran underneath:

```
$ make cocotb
*****
** TEST                                STATUS  SIM TIME (ns)  RATIO **
*****
** test_cordic.test_directed           PASS      1820.00  121098 **
** test_cordic.test_random             PASS      16070.00 128429 **
** test_cordic.test_sweep              PASS      29030.00 128453 **
** test_cordic.test_value_semantics    PASS        230.00  98399 **
*****
** TESTS=4 PASS=4 FAIL=0 SKIP=0        47150.00 127368 **
*****
```

Four tests, four passes, and the third of them is a 181-point sweep of the full circle checked bit for bit against `cordic_model.py`.

Two honest limitations, both confirmed on the machine this book was built on:

- **The SystemVerilog run needs a newer Verilator.** `cocotb 2.1.0` refuses anything before 5.036: `cocotb` requires Verilator 5.036 or later, but using 5.020. Stop. The check is on the Verilator backend only, and it is a version check rather than a missing feature: upgrade and the same test runs.
- **Icarus Verilog cannot compile the package at all**, because of the constant-function ROM. That is not a `cocotb` problem, and `cordic_atan_rom.svh` is the way around it.

```

1  #!/usr/bin/env python3
2  """
3  Generate everything the other testbenches need from the Python golden model.
4
5  This one script demonstrates the two build-time couplings described in
6  Chapter 9:
7
8  (a) *offline vectors* -- a plain text file that any testbench can read,
9  with $readmemh, $fscanf, std.textio or an ifstream;
10
11 (b) *generated source* -- a C++ header and a SystemVerilog include file,
12 both compiled into their respective testbenches.
13
14 Run it from the Makefile, never by hand, so that the generated files can never
15 be older than the model that produced them.
16
17     python3 gen_vectors.py --n 300 --seed 1 \
18         --out ../vectors/cordic_vectors.txt \
19         --header ../verilator/cordic_vectors.h \
20         --svh ../sv/cordic_vectors.svh \
21         --rom ../sv/cordic_atan_rom.svh
22 """
23
24 import argparse
25 import math
26 import os
27 import random
28 import sys
29 import time
30
31 import cordic_model as m
32
33
34 # -----
35 # Stimulus
36 # -----
37 def directed():
38     """The angles a random generator will not hit, and the boundaries of the
39     coarse-rotation stage."""
40     pi_q = m.float_to_q(math.pi, m.QA, m.AW)
41     half = pi_q // 2
42     out = []
43     for th in (0, pi_q, -pi_q, half, -half, half + 1, -half - 1,
44               half // 2, -half // 2, 1, -1):
45         out.append((m.X0_UNIT, 0, th))
46     out.append((0, 0, half)) # zero vector
47     out.append((0, m.X0_UNIT, half)) # start on the y axis
48     out.append((6000, -6000, pi_q // 3))
49     out.append((-6000, 6000, -pi_q // 3))
50     return out
51
52
53 def random_stim(n, seed):
54     rng = random.Random(seed)
55     pi_q = m.float_to_q(math.pi, m.QA, m.AW)
56     return [(rng.randint(-6000, 6000),
57             rng.randint(-6000, 6000),
58             rng.randint(-pi_q, pi_q)) for _ in range(n)]
59
60
61 def build(n, seed):
62     stim = directed() + random_stim(n, seed)
63     rows = []

```

```

64     for x0, y0, th in stim:
65         x, y = m.cordic_rot(x0, y0, th)
66         rows.append((x0, y0, th, x, y))
67     return rows
68
69
70 # -----
71 # (a) the vector file
72 # -----
73 def write_vectors(path, rows, seed):
74     """One transaction per line, 16-bit two's complement in hexadecimal.
75
76     Hexadecimal rather than decimal because $readmemh and $fscanf("%h") both
77     want it, and because a hex field has a fixed width, which makes a short
78     or corrupted line obvious. The header records everything needed to
79     regenerate the file: without it, a vector file is an unreproducible
80     binary blob that happens to be readable.
81     """
82     os.makedirs(os.path.dirname(os.path.abspath(path)), exist_ok=True)
83     with open(path, "w") as f:
84         f.write("# cordic_rot test vectors\n")
85         f.write(f"# generated {time.strftime('%Y-%m-%d %H:%M:%S')}\n")
86         f.write(f"# generator gen_vectors.py seed={seed}\n")
87         f.write(f"# format      DW={m.DW} QF={m.QF} AW={m.AW} QA={m.QA} "
88               f"N={m.N} GL={m.GL} GM={m.GM}\n")
89         f.write(f"# count      {len(rows)}\n")
90         f.write(f"# x0      y0      theta xexp yexp\n")
91         for x0, y0, th, x, y in rows:
92             f.write(f"{x0 & 0xFFFF:04x} {y0 & 0xFFFF:04x} "
93                   f"{th & 0xFFFF:04x} {x & 0xFFFF:04x} {y & 0xFFFF:04x}\n")
94     return len(rows)
95
96
97 # -----
98 # (b) the generated C++ header
99 # -----
100 def write_header(path, rows, seed):
101     with open(path, "w") as f:
102         f.write("// GENERATED by gen_vectors.py -- do not edit.\n")
103         f.write(f"// seed={seed} count={len(rows)}\n")
104         f.write("#ifndef CORDIC_VECTORS_H_ \n#define CORDIC_VECTORS_H_ \n\n")
105         f.write("#include <stdint>\n\n")
106         f.write("struct CordicVec { int16_t x0, y0, theta, x, y; };\n\n")
107         f.write(f"static constexpr unsigned kCordicVecSeed = {seed};\n")
108         f.write(f"static constexpr unsigned kCordicVecCount = {len(rows)};\n\n")
109         f.write("static constexpr CordicVec kCordicVectors[] = {\n\n")
110         for x0, y0, th, x, y in rows:
111             f.write(f"    {{{x0:6d}, {y0:6d}, {th:6d}, {x:6d}, {y:6d}}},\n")
112         f.write(f"};\n\n#endif // CORDIC_VECTORS_H_ \n\n")
113
114
115 # -----
116 # (b) the generated SystemVerilog include
117 # -----
118 def write_svh(path, rows, seed):
119     """An `include file rather than a package, because it is meant to be
120     pulled into an existing testbench package. A standalone table would be
121     better as a package: see the discussion in Chapter 9."""
122     with open(path, "w") as f:
123         f.write("// GENERATED by gen_vectors.py -- do not edit.\n")
124         f.write(f"// seed={seed} count={len(rows)}\n")
125         f.write(f"localparam int unsigned VEC_COUNT = {len(rows)};\n")
126         f.write(f"localparam logic signed [15:0] VEC_X0 [VEC_COUNT] = '{\n\n")

```

```

127     f.write(",\n".join(f" 16'sh{x0 & 0xFFFF:04x}" for x0, _, _, _ in rows))
128     f.write("};\n")
129     f.write("localparam logic signed [15:0] VEC_Y0 [VEC_COUNT] = '{\n")
130     f.write(",\n".join(f" 16'sh{y0 & 0xFFFF:04x}" for _, y0, _, _ in rows))
131     f.write("};\n")
132     f.write("localparam logic signed [15:0] VEC_THETA [VEC_COUNT] = '{\n")
133     f.write(",\n".join(f" 16'sh{t & 0xFFFF:04x}" for _, _, t, _ in rows))
134     f.write("};\n")
135     f.write("localparam logic signed [15:0] VEC_XEXP [VEC_COUNT] = '{\n")
136     f.write(",\n".join(f" 16'sh{x & 0xFFFF:04x}" for _, _, _, x, _ in rows))
137     f.write("};\n")
138     f.write("localparam logic signed [15:0] VEC_YEXP [VEC_COUNT] = '{\n")
139     f.write(",\n".join(f" 16'sh{y & 0xFFFF:04x}" for _, _, _, _, y in rows))
140     f.write("};\n")
141
142
143 # -----
144 # The arctangent ROM, as generated source.
145 #
146 # cordic_pkg.sv computes this table with $atan in a constant function, which
147 # is the elegant way and works in Verilator and in the commercial tools. It
148 # does NOT work in Icarus Verilog, which cannot return an unpacked array from
149 # a function, and some synthesis front ends are equally unhappy. Generating
150 # the table from Python is the portable fallback, and it is the same technique
151 # you use for a register map or a coefficient set.
152 # -----
153 def write_rom(path):
154     with open(path, "w") as f:
155         f.write("// GENERATED by gen_vectors.py -- do not edit.\n")
156         f.write("// arctan(2** -i) in the internal angle format "
157               f"(Q{m.IA} - m.IQA - 1){m.IQA}), i = 0 .. {m.N - 1}\n")
158         f.write(f"localparam iangle_t ATAN_ROM_GEN [{m.N}] = '{\n")
159         last = len(m.ATAN) - 1
160         for i, v in enumerate(m.ATAN):
161             sep = "" if i == last else ", "
162             f.write(f" {m.IA}'sd{v}{sep:<2s}"
163                   f" // atan(2**-{i}) = {math.atan(2.0 ** -i):.9f} rad\n")
164         f.write("};\n")
165
166
167 # -----
168 def main():
169     ap = argparse.ArgumentParser(description=__doc__)
170     ap.add_argument("--n", type=int, default=300, help="random transactions")
171     ap.add_argument("--seed", type=int, default=1)
172     ap.add_argument("--out", default="../vectors/cordic_vectors.txt")
173     ap.add_argument("--header", default=None)
174     ap.add_argument("--svh", default=None)
175     ap.add_argument("--rom", default=None)
176     args = ap.parse_args()
177
178     rows = build(args.n, args.seed)
179
180     n = write_vectors(args.out, rows, args.seed)
181     print(f"wrote {args.out} ({n} vectors, seed {args.seed})")
182
183     if args.header:
184         write_header(args.header, rows, args.seed)
185         print(f"wrote {args.header}")
186     if args.svh:
187         write_svh(args.svh, rows, args.seed)
188         print(f"wrote {args.svh}")
189     if args.rom:

```

```

190     write_rom(args.rom)
191     print(f"wrote {args.rom}")
192
193     # A quick sanity check on the model itself, so that a broken model is
194     # caught here rather than in a simulation three minutes later.
195     worst = 0.0
196     for x0, y0, th, x, y in rows:
197         fx, fy = m.cordic_float(m.q_to_float(x0, m.QF),
198                                m.q_to_float(y0, m.QF),
199                                m.q_to_float(th, m.QA))
200         worst = max(worst,
201                     abs(x - fx * (1 << m.QF)), abs(y - fy * (1 << m.QF)))
202     print(f"model self-check: worst deviation from the ideal rotation "
203           f"= {worst:.2f} LSB")
204     if worst > 8.0:
205         print("model self-check FAILED", file=sys.stderr)
206         return 1
207     return 0
208
209
210 if __name__ == "__main__":
211     sys.exit(main())

```

Listing 1.15. `gen_vectors.py` — vectors, a C++ header, an SystemVerilog include and the ROM, from one model.

```

1  """cocotb testbench for cordic_rot.
2
3  The point of this file is that there is no HDL testbench at all: the top level
4  of the simulation is the DUT itself, and cordic_model.py -- the same module the
5  SystemVerilog and C++ testbenches are checked against -- is imported directly
6  and used as a bit-exact golden model.
7
8  The same file drives the SystemVerilog RTL and the VHDL RTL, with no change
9  at all. Nothing in it is language specific.
10
11     make -f Makefile.cocotb          # VHDL
12     make -f Makefile.cocotb SV=1     # SystemVerilog + Verilator
13 """
14
15 import math
16 import os
17 import random
18 import sys
19 from collections import deque
20 from pathlib import Path
21
22 import cocotb
23 from cocotb.clock import Clock
24 from cocotb.triggers import ClockCycles, RisingEdge, with_timeout
25
26 sys.path.insert(0, str(Path(__file__).resolve().parent))
27 import cordic_model as m
28
29 TOL_LSB = 12
30
31
32 class Txn:
33     __slots__ = ("x0", "y0", "theta", "x", "y")
34
35     def __init__(self, x0, y0, theta):
36         self.x0, self.y0, self.theta = x0, y0, theta
37         self.x = self.y = None
38
39     def __repr__(self):
40         return f"Txn(x0={self.x0}, y0={self.y0}, theta={self.theta})"
41
42
43 async def reset(dut, cycles=5):
44     dut.rst_ni.value = 0
45     dut.req_valid_i.value = 0
46     dut.rsp_ready_i.value = 1
47     dut.req_x_i.value = 0
48     dut.req_y_i.value = 0
49     dut.req_theta_i.value = 0
50     await ClockCycles(dut.clk_i, cycles)
51     dut.rst_ni.value = 1
52     await RisingEdge(dut.clk_i)
53
54
55 async def driver(dut, queue, stimulus):
56     """Push transactions into the request channel, honouring req_ready_o."""
57     for txn in stimulus:
58         await RisingEdge(dut.clk_i)
59         dut.req_x_i.value = txn.x0 & 0xFFFF
60         dut.req_y_i.value = txn.y0 & 0xFFFF
61         dut.req_theta_i.value = txn.theta & 0xFFFF
62         dut.req_valid_i.value = 1
63         while True:

```

```

64         await RisingEdge(dut.clk_i)
65         if dut.req_ready_o.value == 1:
66             break
67         dut.req_valid_i.value = 0
68         queue.append(txn)
69
70
71 async def monitor(dut, queue, results, expected):
72     """Sample the response channel on every accepted handshake."""
73     while len(results) < expected:
74         await RisingEdge(dut.clk_i)
75         if dut.rsp_valid_o.value == 1 and dut.rsp_ready_i.value == 1:
76             txn = queue.popleft()
77             txn.x = dut.rsp_x_o.value.to_signed()
78             txn.y = dut.rsp_y_o.value.to_signed()
79             results.append(txn)
80
81
82 def scoreboard(results, log):
83     """Bit-exact check against cordic_model, plus a float sanity check."""
84     errors = 0
85     worst = 0.0
86     for txn in results:
87         ex, ey = m.cordic_rot(txn.x0, txn.y0, txn.theta)
88         if (txn.x, txn.y) != (ex, ey):
89             errors += 1
90             if errors <= 5:
91                 log.error("%r: got (%d, %d) expected (%d, %d)",
92                           txn, txn.x, txn.y, ex, ey)
93         fx, fy = m.cordic_float(txn.x0, txn.y0, txn.theta / (1 << m.QA))
94         worst = max(worst, abs(txn.x - fx), abs(txn.y - fy))
95     return errors, worst
96
97
98 async def run(dut, stimulus):
99     cocotb.start_soon(Clock(dut.clk_i, 10, unit="ns").start())
100    await reset(dut)
101
102    queue, results = deque(), []
103    mon = cocotb.start_soon(monitor(dut, queue, results, len(stimulus)))
104    drv = cocotb.start_soon(driver(dut, queue, stimulus))
105    await with_timeout(drv, 200 * len(stimulus) + 1000, "ns")
106    await with_timeout(mon, 200 * len(stimulus) + 1000, "ns")
107
108    errors, worst = scoreboard(results, dut._log)
109    dut._log.info("%d transactions, %d bit-exact mismatches, "
110                 "worst float error %.2f LSB", len(results), errors, worst)
111    assert errors == 0, f"{errors} bit-exact mismatches"
112    assert worst <= TOL_LSB, f"worst float error {worst:.2f} > {TOL_LSB} LSB"
113
114
115 @cocotb.test()
116 async def test_directed(dut):
117     """Angles a random generator will not hit."""
118     angs = [0.0, math.pi / 2, -math.pi / 2, math.pi / 4, -math.pi / 4,
119            1e-4, -1e-4, 3.14, -3.14]
120     stim = [Txn(m.X0_UNIT, 0, m.float_to_q(a, m.QA, m.AW)) for a in angs]
121     stim += [Txn(0, 0, 0), Txn(6000, -6000, 0)]
122     await run(dut, stim)
123
124
125 @cocotb.test()
126 async def test_random(dut):

```

```

127     """Constrained random stimulus, seeded from the environment."""
128     seed = int(os.environ.get("SEED", "20250905"))
129     rng = random.Random(seed)
130     lim = m.float_to_q(math.pi, m.QA, m.AW)
131     stim = [Txn(rng.randint(-6000, 6000), rng.randint(-6000, 6000),
132                rng.randint(-lim, lim)) for _ in range(100)]
133     dut._log.info("seed = %d", seed)
134     await run(dut, stim)
135
136
137 @cocotb.test()
138 async def test_sweep(dut):
139     """Full sweep of the unit circle: this is the accuracy test."""
140     n = 181
141     stim = [Txn(m.X0_UNIT, 0,
142                m.float_to_q(-math.pi + 2 * math.pi * k / (n - 1),
143                m.QA, m.AW))
144            for k in range(n)]
145     await run(dut, stim)
146
147
148 @cocotb.test()
149 async def test_value_semantics(dut):
150     """What .value actually gives you on a multi-bit signed port."""
151     cocotb.start_soon(Clock(dut.clk_i, 10, unit="ns").start())
152     await reset(dut)
153     await run_one(dut, m.X0_UNIT, 0, m.float_to_q(math.pi, m.QA, m.AW))
154     v = dut.rsp_x_o.value
155     # cocotb 2.x spelling; in 1.x these were v.binstr, v.integer and
156     # v.signed_integer. The trap is the same in both: the plain conversion
157     # is UNSIGNED, so a negative result comes back as a large positive one.
158     dut._log.info("str(v)          = %s", str(v))
159     dut._log.info("int(v)          = %d", int(v))
160     dut._log.info("v.to_unsigned()= %d", v.to_unsigned())
161     dut._log.info("v.to_signed()  = %d", v.to_signed())
162     dut._log.info("model says     = %d",
163                m.cordic_rot(m.X0_UNIT, 0,
164                m.float_to_q(math.pi, m.QA, m.AW))[0])
165
166
167 async def run_one(dut, x0, y0, th):
168     txn = Txn(x0, y0, th)
169     q, res = deque(), []
170     mon = cocotb.start_soon(monitor(dut, q, res, 1))
171     drv = cocotb.start_soon(driver(dut, q, [txn]))
172     await with_timeout(drv, 5000, "ns")
173     await with_timeout(mon, 5000, "ns")

```

Listing 1.16. `test_cordic.py` — the cocotb testbench. The same file drives the VHDL and the SystemVerilog.

1.12 Build and Run

Listing 1.17 is the whole build. Six ways of exercising one design, and `make all` runs the five that need only free tools. The two VHDL targets, `make vhd1` and `make cocotb`, are separate because they need a VHDL simulator; the command lines in the Makefile are QuestaSim’s.

Table 1.4. What checks what. “The VHDL flow” is `vcom/vsim` on a commercial simulator, as Section 1.8 sets it out. The last two rows are what tie the two implementations together.

Testbench	Lang.	Simulator	Reference	Check
<code>tb_cordic_rot.vhd</code>	VHDL	the VHDL flow	<code>ieee.math_real</code>	tolerance
<code>tb_cordic_top.sv</code>	SV	Verilator	<code>\$sin/\$cos</code>	tolerance
<code>tb_cordic.cpp</code>	C++	Verilator	<code><cmath></code>	tolerance
<code>main_uvmlike.cpp</code>	C++	Verilator	<code><cmath></code>	tolerance
<code>tb_vectors.cpp</code>	C++	Verilator	<code>cordic_model.py</code>	bit-exact
<code>test_cordic.py</code>	Python	the VHDL flow	<code>cordic_model.py</code>	bit-exact

The verification argument is worth stating explicitly, because it is the reason for the last two rows:

1. The SystemVerilog implementation is bit-exact against the Python model: `make bitexact`, 315 vectors, 0 mismatches.
2. The VHDL implementation is bit-exact against the *same* Python model: `make cocotb`, four tests, 0 mismatches, including a 181-point sweep of the full circle.
3. Therefore the two implementations agree with each other, bit for bit, on every vector either of them has seen.

That is a stronger statement than “both testbenches pass”, and it costs one extra script. Note also what the argument does *not* depend on. Neither step appeals to a particular simulator: step 1 needs some tool that runs the SystemVerilog, step 2 needs some tool that runs the VHDL, and the transitive conclusion follows from the two comparisons against the one model. Change either backend and the chain still closes.

There is one more target, and it produces something to look at rather than a pass or a fail. `make waves` rebuilds the SystemVerilog testbench with tracing enabled and runs a short twenty-transaction stimulus, leaving `tb_cordic.vcd` behind. A VCD is a file, not a viewer: open it in GTKWave or in Surfer, or read it into a commercial simulator’s own wave window. The *Simulation Cookbook* covers tracing properly, including why you should name the scope you dump rather than dumping the whole hierarchy.

Recipe: Lint first, every time

`make lint` is `verilator -lint-only -Wall` over the synthesisable sources, and it fails the build. In VHDL the analyser does this job for free. In SystemVerilog it is a target you have to write and a habit you have to acquire, and it is the single highest-value line in the Makefile.

```

# =====
# Makefile for the CORDIC example.
#
# Targets
# make lint      Verilator lint of the synthesisable SystemVerilog
# make vhdل      analyse, elaborate and run the VHDL testbench (QuestaSim)
# make sv        run the SystemVerilog class testbench (Verilator)
# make cpp       run the plain C++ testbench (Verilator)
# make uvmlike   run the UVM-shaped C++ environment (Verilator)
# make vectors   regenerate the stimulus/expected vectors with Python
# make cocotb    run the cocotb testsuite
# make waves     as `sv', with a VCD
# make all       everything
# make clean
# =====

SHELL      := /bin/bash
ROOT       := $(abspath $(dir $(lastword $(MAKEFILE_LIST))))

SV_DIR     := $(ROOT)/sv
VHDL_DIR   := $(ROOT)/vhdل
VL_DIR     := $(ROOT)/verilator
PY_DIR     := $(ROOT)/python
VEC_DIR    := $(ROOT)/vectors
BUILD      := $(ROOT)/build

RTL_SV     := $(SV_DIR)/cordic_pkg.sv $(SV_DIR)/cordic_rot.sv
TB_SV      := $(SV_DIR)/cordic_if.sv $(SV_DIR)/tb_cordic_pkg.sv \
            $(SV_DIR)/tb_cordic_top.sv
RTL_VHDL   := $(VHDL_DIR)/cordic_pkg.vhd $(VHDL_DIR)/cordic_rot.vhd
TB_VHDL    := $(VHDL_DIR)/tb_cordic_rot.vhd

N_RANDOM   ?= 300
SEED       ?= 1

VERILATOR  ?= verilator
VCOM       ?= vcom
VSIM       ?= vsim
VLIB       ?= vlib
PYTHON     ?= python3

# Warnings that are switched off on purpose; each one is discussed in the
# book. A real project keeps this list short and justifies every entry.
VL_WAIVE   := -Wno-CONSTRAINTIGN -Wno-WIDTHEXPAND -Wno-WIDTHTRUNC \
            -Wno-UNUSED SIGNAL -Wno-DECLFILENAME

.PHONY: all lint vhdل sv cpp uvmlike bitexact vectors model cocotb cocotb-vhdل cocotb-sv
↳ waves clean

# `all' deliberately leaves out the VHDL and cocotb targets, which need
# tools that may not be installed. Run them explicitly.
all: lint sv cpp uvmlike bitexact

# -----
# Lint. Run this before anything else, every time. In SystemVerilog the
# linter does the job VHDL's type checker does for free.
# -----
lint:
    $(VERILATOR) --lint-only -Wall -Wno-DECLFILENAME $(RTL_SV) \
        --top-module cordic_rot
    @echo "lint: clean"

# -----

```

```

# VHDL
# -----
# Needs a VHDL simulator; the commands below are QuestaSim/ModelSim.
vhdl:
  @mkdir -p $(BUILD)/vhdl
  cd $(BUILD)/vhdl && \
    $(VLIB) work && \
    $(VCOM) -2008 -quiet $(RTL_VHDL) $(TB_VHDL) && \
    $(VSIM) -c -gN_RANDOM=$(N_RANDOM) \
      -do "set NumericStdNoWarnings 1; run -all; quit" tb_cordic_rot

# -----
# SystemVerilog class testbench, run under Verilator.
#
# Note what is lost: no constraint solver, no covergroups, and the clocking
# blocks are compiled out (see the shim at the top of tb_cordic_pkg.sv).
# Under QuestaSim the same sources run with all three.
# -----
sv:
  @mkdir -p $(BUILD)/obj_sv
  $(VERILATOR) --binary --timing -j 0 $(VL_WAIVE) \
    -Mdir $(BUILD)/obj_sv -o Vtb \
    $(RTL_SV) $(TB_SV) --top-module tb_cordic_top
  $(BUILD)/obj_sv/Vtb +NRANDOM=$(N_RANDOM) +SEED=$(SEED)

waves:
  @mkdir -p $(BUILD)/obj_wav
  $(VERILATOR) --binary --timing --trace -j 0 $(VL_WAIVE) \
    -Mdir $(BUILD)/obj_wav -o Vtb \
    $(RTL_SV) $(TB_SV) --top-module tb_cordic_top
  $(BUILD)/obj_wav/Vtb +NRANDOM=20 +WAVES
  @echo "wrote tb_cordic.vcd"

# -----
# Plain C++ testbench
# -----
cpp:
  @mkdir -p $(BUILD)/obj_cpp
  $(VERILATOR) --cc --exe --build --trace -j 0 $(VL_WAIVE) \
    -Mdir $(BUILD)/obj_cpp -o Vcpp \
    $(RTL_SV) $(VL_DIR)/tb_cordic.cpp --top-module cordic_rot
  $(BUILD)/obj_cpp/Vcpp

# -----
# UVM-shaped C++ environment
# -----
uvmlike:
  @mkdir -p $(BUILD)/obj_uvm
  $(VERILATOR) --cc --exe --build -j 0 $(VL_WAIVE) \
    -Mdir $(BUILD)/obj_uvm -o Vuvmlike \
    -CFLAGS "-I$(VL_DIR) -std=c++17" \
    $(RTL_SV) $(VL_DIR)/main_uvmlike.cpp --top-module cordic_rot
  $(BUILD)/obj_uvm/Vuvmlike

# -----
# Bit-exact check against the Python model, using generated vectors
# -----
bitexact: vectors
  @mkdir -p $(BUILD)/obj_vec
  $(VERILATOR) --cc --exe --build -j 0 $(VL_WAIVE) \
    -Mdir $(BUILD)/obj_vec -o Vvec \
    -CFLAGS "-I$(VL_DIR) -std=c++17" \
    $(RTL_SV) $(VL_DIR)/tb_vectors.cpp --top-module cordic_rot

```

```

$(BUILD)/obj_vec/Vvec

# -----
# Python
# -----
vectors:
    @mkdir -p $(VEC_DIR)
    $(PYTHON) $(PY_DIR)/gen_vectors.py --n $(N_RANDOM) --seed $(SEED) \
        --out $(VEC_DIR)/cordic_vectors.txt \
        --header $(VL_DIR)/cordic_vectors.h \
        --svh $(SV_DIR)/cordic_vectors.svh \
        --rom $(SV_DIR)/cordic_atan_rom.svh

model:
    $(PYTHON) $(PY_DIR)/cordic_model.py

# The same Python testbench, against both implementations. The SV run needs
# Verilator 5.036 or later (cocotb refuses anything older) or a commercial
# simulator; the VHDL run needs a VHDL simulator.
cocotb: cocotb-vhdl

cocotb-vhdl:
    $(MAKE) -C $(PY_DIR) -f Makefile.cocotb clean
    $(MAKE) -C $(PY_DIR) -f Makefile.cocotb

cocotb-sv:
    $(MAKE) -C $(PY_DIR) -f Makefile.cocotb clean
    $(MAKE) -C $(PY_DIR) -f Makefile.cocotb SV=1

# -----
clean:
    rm -rf $(BUILD) $(PY_DIR)/sim_build $(PY_DIR)/__pycache__ \
        $(PY_DIR)/results.xml *.vcd

```

Listing 1.17. The build. `make all` runs lint, the SystemVerilog class testbench, both C++ testbenches and the bit-exact check; `make vhdl` and `make cocotb` are separate because they need a VHDL simulator.

1.13 What Is Not Verified

Six testbenches and a green regression are not the same thing as a verified design. What has been established is that the RTL computes the right arithmetic on the vectors it has seen. Here is what has not, roughly in the order a real project would address it.

1. **Coverage is not closed.** The covergroups do not run under Verilator, so the only coverage number in this chapter is the hand-rolled one in the C++ environment, and it reports a hole. Nobody has measured which states, which transitions or which angle ranges the 300 random transactions actually exercised. Run the SystemVerilog testbench under a simulator with covergroups and close the holes; that is the first thing to do.
2. **The saturating output is barely tested.** Two directed vectors reach it. There is no test that walks the input magnitude across the saturation boundary, and none at all for the negative limit, where an off-by-one is most likely.
3. **Reset is tested once.** Reset is asserted at time zero and released. There is no test that asserts it mid-transaction, none that checks recovery, and none that varies its release relative to the clock edge.
4. **The protocol is under-checked.** One concurrent assertion covers request stability.

Nothing checks that `rsp_valid_o` stays asserted until `rsp_ready_i`, that the response payload is stable while stalled, or that no response appears without a request. Those are three more `assert` property statements and they belong in a `bind`-attached checker.

5. **No gate-level simulation.** The design has not been synthesised. Verilator is two-state, so a reset bug that depends on `x` propagation would not show up here at all — see the `-x-assign` discussion in the *Simulation Cookbook*.
6. **No formal.** The handshake properties are exactly the kind of thing a bounded model checker proves in seconds. `assume` the input protocol, `assert` the output protocol, and you get a proof rather than 308 samples.
7. **One parameterisation.** Everything was run at `ITER = 14`. The parameter exists; nothing sweeps it.

A regression that passes tells you what you tested. Write down what you did not.

1.14 Exercises

1. **Sweep ITER.** Run the testbench at `ITER = 8, 10, 12, 14` and `16` and plot the worst error. Explain the floor. *Teaches: where the error budget actually comes from.*
2. **Break it on purpose.** Change `>>` to `>` in `cordic_rot.sv` and run every testbench. Which ones fail, how loudly, and how long does each take to point you at the line? *Teaches: what a scoreboard is for.*
3. **Make it build under Icarus.** Add a `'ifdef` to `cordic_pkg.sv` that uses the generated `cordic_atan_rom.svh` instead of the constant function, and get `iverilog -g2012` to compile the design. *Teaches: generated source, and why portability costs something.*
4. **Add the missing protocol checks.** Write the three assertions listed in Section 1.13 in a separate checker module and attach it with `bind`, without editing `cordic_rot.sv`. *Teaches: bind, and assertion-based verification of code you do not own.*
5. **Close the coverage.** Add a covergroup that crosses the angle quadrant with the sign of y_0 and with whether the output saturated, and add stimulus until it closes. *Teaches: the coverage-driven loop.*
6. **Vectoring mode.** Change the sign decision from $d_i = \text{sign}(z_i)$ to $d_i = -\text{sign}(y_i)$ and the block computes $\sqrt{x^2 + y^2}$ and $\arctan(y/x)$ instead. Update the model first, then the RTL. *Teaches: that the golden model is the specification.*
7. **Pipeline it.** Unroll the loop into N stages so the block accepts a transaction every cycle. Work out what happens to `req_ready_o` and to `rsp_valid_o`, and what the monitor has to do differently. *Teaches: that the interesting part of a pipeline is the handshake, not the datapath.*
8. **Rewrite the testbench in UVM.** Take `tb_cordic_pkg.sv` and map each class onto its UVM base, using the table in the *Verification Cookbook*. Count the lines before and after. *Teaches: exactly what UVM adds, and what it costs.*

1.15 Checklist

- Write the number format down before the RTL, and put it in the package. SystemVerilog will not check it for you.
- Write the golden model before the RTL, from the specification, and never by reading the

RTL.

- Make the model bit-exact if the design is fixed-point. A tolerance check and a bit-exact check catch different bugs; run both.
- Compute constant tables at elaboration where the tools allow it, and generate them from a script where they do not. Put a **GENERATED** banner in the first line of anything generated.
- Put `make lint` first in the build and make it fail the build. It is the replacement for VHDL's analyser.
- Use `>>` on signed values. Check every shift in the design before you check anything else.
- Reach the DUT through a clocking block. Where a tool cannot, say so in a comment at the top of the file and keep one set of sources.
- End the test by draining and timing out, never by waiting a fixed number of cycles.
- Send the corners you can name; use coverage for the ones you cannot.
- Run two simulators if you can. The bugs that cost a day are the ones one tool implements silently wrong.
- Write down what you did not verify, in the same commit as the testbench that passes.

Further Reading

Standards

- IEEE Std 1800-2017, *IEEE Standard for SystemVerilog — Unified Hardware Design, Specification, and Verification Language*. The language reference manual. Chapter 9 (processes), Chapter 10 (assignment statements) and Chapter 4 (scheduling semantics) are the three a VHDL designer should read first.
- IEEE Std 1076-2008, *IEEE Standard VHDL Language Reference Manual*, and IEEE Std 1076-2019 for the newer features discussed in the *Design Cookbook*.
- IEEE Std 1800.2-2020, *Universal Verification Methodology Language Reference Manual*.
- IEEE Std 1364-2005, the last standalone Verilog standard, useful only when reading legacy code.

Books

- C. Spear and G. Tumbush, *SystemVerilog for Verification*, 3rd ed., Springer. The standard text on the class-based testbench, and the natural sequel to the *Verification Cookbook*.
- S. Sutherland, S. Davidmann and P. Flake, *SystemVerilog for Design*, 2nd ed., Springer. The design-side counterpart, written by people who were in the room when the language was standardised.
- D. Thomas, *The Verilog Hardware Description Language*, and P. Ashenden, *The Designer's Guide to VHDL*. The two classics, for when you want to know why something is the way it is.
- P. Ashenden and J. Lewis, *VHDL-2008: Just the New Stuff*. Short, and it covers most of what the *Design Cookbook* says VHDL can do.
- M. Keating, *The Simple Art of SoC Design*. Not about either language, and worth reading anyway.

Papers and application notes

- C. Cummings, *Nonblocking Assignments in Verilog Synthesis, Coding Styles That Kill!*, SNUG 2000. The clearest published explanation of the blocking/non-blocking rules and the races they exist to prevent. Everything else by the same author on `always_comb`, `unique/priority` and clock-domain crossing is worth the time.
- J. E. Volder, *The CORDIC Trigonometric Computing Technique*, IRE Transactions on Electronic Computers, 1959. The original paper behind Chapter 1.
- J. S. Walther, *A Unified Algorithm for Elementary Functions*, Spring Joint Computer Conference, 1971. The generalisation.
- R. Andraka, *A Survey of CORDIC Algorithms for FPGA Based Computers*, FPGA 1998. The practical implementation reference, including the gain, the convergence range and the quadrant folding used in Chapter 1.

Tools and online resources

- *Verilator* manual, <https://verilator.org/guide/>. Read the chapters on the language subset and on the C++ API before writing a C++ testbench.
- *cocotb* documentation, <https://docs.cocotb.org/>.
- *GTKWave* (<https://gtkwave.sourceforge.net>) and *Surfer* (<https://surfer-project.org>), the two waveform viewers that read VCD and FST.
- *OSVVM* (<https://osvvm.org>) and *UVVM* (<https://uvvm.org>), the two VHDL verification frameworks referred to throughout the *Design Cookbook* and the *Verification Cookbook*.
- The *lowRISC* and *OpenTitan* SystemVerilog style guide, <https://github.com/lowRISC/style-guides>. The best free statement of what disciplined SystemVerilog looks like, and the source of the naming conventions used in this book.
- *Verible*, <https://github.com/chipsalliance/verible>, for formatting and linting.

Index

CORDIC, [1](#)
 rotation mode, [2](#)
golden model, [6](#)